

```

#include <windows.h>

#include <d3d11.h>

#include <d3dcompiler.h>

#include <cmath> // ИСПРАВЛЕНИЕ: Подключили математические функции sqrt и pow для CPU

#include <commctrl.h> // Системная библиотека элементов управления Windows

#pragma comment(lib, "comctl32.lib") // Автоматическая линковка библиотеки

#pragma comment(lib, "d3d11.lib")

#pragma comment(lib, "d3dcompiler.lib")


// === НОВЫЙ КОД: ИДЕНТИФИКАТОРЫ ДЛЯ ВЕРХНЕГО МЕНЮ WINDOWS ===

#define IDM_FILE_OPEN_VIDEO 1001 // Открыть видеофайл (AVI/MP4)

#define IDM_FILE_OPEN_IMAGE 1002 // Открыть отдельное изображение (BMP/PNG)

#define IDM_FILE_SAVE_RESULT 1003 // Записать преобразованный файл

#define IDM_FILE_EXIT 1004 // Выход из программы


#define IDM_VIEW_SPLIT 1005 // Режим "До / После" (разделение экрана)

#define IDM_VIEW_RESULT_ONLY 1006 // Режим "Только результат"


#define IDM_HELP_ABOUT 1007 // О программе

#define IDM_VIEW_CPU 40003


// Идентификаторы для дискретных масштабов дефектоскопа

#define IDM_ZOOM_FIT 40010

#define IDM_ZOOM_1 40011

#define IDM_ZOOM_2 40012

#define IDM_ZOOM_4 40013

#define IDM_ZOOM_8 40014

#define IDM_ZOOM_16 40015


// Структура вершины для нашего видеоэкрана

struct SimpleVertex

{

    float Pos[3]; // Координаты X, Y, Z в пространстве экрана

```

```

float Tex[2]; // Текстурные координаты U, V для видеопотока
};

// Структура параметров кадра для передачи в шейдеры
struct ShaderConstants
{
    float width; // Ширина картинки
    float height; // Высота картинки
    float d_width; // Шаг одного пикселя по горизонтали (1.0 / width)
    float d_height; // Шаг одного пикселя по вертикали (1.0 / height)
    float splitX; // Положение шторки (доля от 0.0 до 1.0)
    float padding; // Сюда мы запишем наш полупиксельный сдвиг!
    float macroWidth; // Сюда запишем ширину макротекстуры (46.0f)
    float macroHeight; // Сюда запишем высоту макротекстуры (46.0f)
};

// === ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ===
HWND hwnd;

// Размеры оригинального кадра
UINT g_srcImgWidth = 0;
UINT g_srcImgHeight = 0;

// Размеры кадра, дополненного до кратности 8
UINT g_paddedImgWidth = 0;
UINT g_paddedImgHeight = 0;

// Размеры сжатого кадра (Режим 3)
UINT g_compressedWidth = 0;
UINT g_compressedHeight = 0;

// 4 главных буфера данных (каждый пиксель - 32-битный DWORD: 0x00RRGGBB)

```

```

DWORD* g_pBuffer1_Src = nullptr;    // Исходный кадр

DWORD* g_pBuffer2_Padded = nullptr;  // Дополненный до кратности 8

DWORD* g_pBuffer3_Monochrome = nullptr; // Монохромный высокого разрешения

DWORD* g_pBuffer4_Compressed8x = nullptr; // Сжатый в 8 раз (Дефектоскоп)


// === ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ДЛЯ НЕЗАВИСИМОГО ДИАГНОСТИЧЕСКОГО ОКНА ===

HWND      g_hDlgWnd = NULL;          // Дескриптор диагностического окна Windows
HWND      g_hStatusWnd = NULL;       // Дескриптор строки статуса внизу плеера
HWND      g_hDlgStatusWnd = NULL;     // Строка статуса диагностического окна панели
IDXGISwapChain* g_pDlgSwapChain = NULL; // Цепочка буферов для второго экрана
ID3D11RenderTargetView* g_pDlgRenderTargetView = NULL; // Цель отрисовки диагностического
окна

void CreateDiagnosticWindow(HINSTANCE hInstance, HWND hParentWnd);

int      g_diagMode = 1;              // Текущий режим панели: 1, 2, 3 или 4

ID3D11Texture2D* g_pMacroStaging = NULL; // Промежуточная текстура для передачи
данных из GPU в CPU

ID3D11Texture2D* g_pTextureStaging = NULL; // Промежуточная текстура для оригинала
кадра


UINT* pPixelsBuffer = NULL;           // Наш глобальный массив кадра для CPU и отладки!
UINT* pRawPixels = NULL;              // Наш глобальный массив кадра для CPU и отладки!
UINT* pCompressedBuffer8x = NULL;     // Буфер для хранения уменьшенной копии кадра
на CPU

UINT* pStretchedBuffer = NULL;        // Буфер для хранения кадра, растянутого обратно
после сжатия

// =====

bool g_bNeedsUpdate = true;           // По умолчанию true, чтобы плеер отрисовал самый
первый кадр при старте

HMENU g_hZoomSubMenu = NULL;          // Глобальный дескриптор для подменю
масштабов

void UpdateZoomMenuCheck(int checkedId);


// Прототип функции программной фильтрации на центральном процессоре

HRESULT ApplyCpuFilter(unsigned int* pSrcPixels, unsigned int width, unsigned int height, unsigned int*
pOutPixels);

```

// Глобальный указатель на буфер констант

ID3D11Buffer* g_pConstantBuffer = NULL;

// === НОВЫЙ КОД: ОБЪЕКТЫ ДЛЯ СВЕРТКИ АНАЛИЗА БЛОКОВ 4x4 ===

ID3D11RenderTargetView* g_pStage1RTV = NULL; // Холст для записи фиолетового анализа 3-го шейдера

ID3D11ShaderResourceView* g_pStage1SRV = NULL; // Ссылка на этот анализ для следующего шейдера

// === НОВЫЙ КОД: ОБЪЕКТЫ ДЛЯ УКРУПНЕННОГО АНАЛИЗА МАКРОБЛОКОВ 8x8 ===

ID3D11RenderTargetView* g_pStage2RTV = NULL; // Холст для записи макроанализа 4-го шейдера

ID3D11ShaderResourceView* g_pStage2SRV = NULL; // Ссылка на макроанализ для последующих шейдеров

// ОБЪЕКТЫ ДЛЯ ФИЛЬТРА ГЛОБАЛЬНОЙ КОНТРАСТНОСТИ (5-й ШЕЙДЕР)

ID3D11RenderTargetView* g_pStage3RTV = NULL; // Холст для записи HDR-фильтра

ID3D11ShaderResourceView* g_pStage3SRV = NULL; // Ссылка на этот результат

// === НОВЫЙ КОД: МАСТЕР-БУФЕР ДЛЯ ПОЛНОРАЗМЕРНОЙ СКЛЕЙКИ ШТОРКИ ===

ID3D11RenderTargetView* g_pStageMasterRTV = NULL; // Холст для записи полноразмерной шторки

ID3D11ShaderResourceView* g_pStageMasterSRV = NULL; // Ссылка на мастер-кадр для последующего сжатия

// === ТЕКСТУРНЫЕ БУФЕРЫ ДЛЯ СТАДИЙ ШЕЙДЕРНОГО СЖАТИЯ ===

ID3D11PixelShader* g_pPixelShaderDownsample = NULL; // Указатель на новый шейдер сжатия

// Стадия 1:2

ID3D11RenderTargetView* g_pStageDown2RTV = NULL;

ID3D11ShaderResourceView* g_pStageDown2SRV = NULL;

// Стадия 1:4

ID3D11RenderTargetView* g_pStageDown4RTV = NULL;

```
ID3D11ShaderResourceView* g_pStageDown4SRV = NULL;
```

```
// Стадия 1:8
```

```
ID3D11RenderTargetView* g_pStageDown8RTV = NULL;
```

```
ID3D11ShaderResourceView* g_pStageDown8SRV = NULL;
```

```
// Стадия 1:16
```

```
ID3D11RenderTargetView* g_pStageDown16RTV = NULL;
```

```
ID3D11ShaderResourceView* g_pStageDown16SRV = NULL;
```

```
// === НОВЫЕ ПЕРЕМЕННЫЕ ДЛЯ ПРОГРАММНОГО РЕЖИМА (CPU) ===
```

```
bool          g_bUseCPUProcessing = false; // true - процессор, false - видеокарта
```

```
ID3D11Texture2D* g_pCpuTexture = NULL;      // Промежуточный холст для пикселей от CPU
```

```
ID3D11RenderTargetView* g_pMacroRTV4x4 = NULL;    // Сюда Пасс А будет записывать блоки
```

```
ID3D11ShaderResourceView* g_pMacroSRV4x4 = NULL;  // Отсюда Пасс Б (шейдер) будет читать  
блоки
```

```
ID3D11ShaderResourceView* g_pCpuTextureSRV = NULL; // Ресурс чтения результата CPU для  
шторки
```

```
// === ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ДЛЯ ПОЛОС ПРОКРУТКИ ===
```

```
int          g_scrollX = 0; // Текущий сдвиг картинки по горизонтали в пикселях
```

```
int          g_scrollY = 0; // Текущий сдвиг картинки по вертикали в пикселях
```

```
int          g_dlgScrollX = 0; // Сдвиг по горизонтали для окна диагностики
```

```
int          g_dlgScrollY = 0; // Сдвиг по вертикали для окна диагностики
```

```
int          g_dlgScrollX_8x8 = 0; // Сдвиг по горизонтали для окна диагностики
```

```
int          g_dlgScrollY_8x8 = 0; // Сдвиг по вертикали для окна диагностики
```

```
UINT         g_currentImgWidth = 0; // Храним размеры текущего файла
```

```
UINT         g_currentImgHeight = 0;
```

```
// === ГЛОБАЛЬНЫЕ РАЗМЕРЫ КЛИЕНТСКОЙ ОБЛАСТИ ОКНА ===
```

```
float        g_wndW = 800.0f; // Стартовая ширина окна по умолчанию
```

```
float        g_wndH = 600.0f; // Стартовая высота окна по умолчанию
```

```
D3D11_VIEWPORT vpWin = { 0.0f, 0.0f, 800.0f, 600.0f, 0.0f, 1.0f };
```

```
// === ГЛОБАЛЬНЫЕ ДОЛИ ПРОКРУТКИ КАДРА (ОТ 0.0 ДО 1.0) ===
```

```
float          g_scrollRatioX = 0.0f;
```

```
float          g_scrollRatioY = 0.0f;
```

```
// Глобальный делитель масштаба кадра
```

```
float          g_zoomDivider = 1.0f;
```

```
// Глобальный флаг режима автоматического вписывания картинки в окно
```

```
bool           g_bFitToWindow = true; // По умолчанию плеер стартует в режиме "Вписать в кадр"!
```

```
// === ГЛОБАЛЬНЫЕ МАСШТАБНЫЕ МНОЖИТЕЛИ ДЛЯ ИДЕАЛЬНОГО СКРОЛЛИНГА ===
```

```
float          g_scrollMultiplierX = 1.0f;
```

```
float          g_scrollMultiplierY = 1.0f;
```

```
// === НОВЫЙ КОД: ПЕРЕМЕННАЯ ДЛЯ ХРАНЕНИЯ ПУТИ К ОТКРЫТОМУ ФАЙЛУ ===
```

```
wchar_t        g_szSelectedFilePath[MAX_PATH] = L""; // Буфер для пути к картинке или видео
```

```
// --- ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ (СТРОГО ПО ОДНОМУ ЭКЗЕМПЛЯРУ): ---
```

```
ID3D11Device* g_pd3dDevice = NULL;    // Видеокарта
```

```
ID3D11DeviceContext* g_pImmediateContext = NULL; // Контекст команд
```

```
IDXGISwapChain* g_pSwapChain = NULL;    // Буфер экрана
```

```
ID3D11RenderTargetView* g_pRenderTargetView = NULL; // Окно вывода
```

```
ID3D11VertexShader* g_pVertexShader = NULL; // Вершинный шейдер
```

```
ID3D11PixelShader* g_pPixelShader = NULL;    // Пиксельный шейдер
```

```
ID3D11PixelShader* g_pPixelShaderStage2 = NULL; // НОВОЕ: Указатель на 4-й шейдер анализа макроблоков 8x8
```

```
ID3D11PixelShader* g_pPixelShaderStage3 = NULL; // НОВОЕ: Указатель на 5-й шейдер адаптивной фильтрации контраста
```

```
ID3D11PixelShader* g_pPixelShaderFinal = NULL; // НОВОЕ: Указатель на 6-й шейдер финальной цветокоррекции и гаммы
```

```
ID3D11PixelShader* g_pPixelShaderCopy = NULL; // Указатель на новый чистый шейдер
копирования кадра
```

```
ID3D11InputLayout* g_pVertexLayout = NULL; // Формат вершин
```

```
ID3D11Buffer* g_pVertexBuffer = NULL; // Буфер геометрии
```

```
SimpleVertex g_currentVertices[4]; // Массив вершин, хранящий живую геометрию скроллинга
окна
```

```
ID3D11ShaderResourceView* g_pTextureSRV = NULL; // Наша текстура 2x2
```

```
ID3D11Texture2D* g_pMacroTexture4x4 = NULL; // Сам холст 46x46 в видеопамяти
```

```
ID3D11SamplerState* g_pSamplerState = NULL; // Жесткий сэмплер (Point)
```

```
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam);
```

```
void CleanupDevice(); // НОВОЕ: Объявляем функцию очистки заранее, чтобы убрать ошибку
компиляции
```

```
// Функция для компиляции HLSL-файлов
```

```
HRESULT CompileShaderFromFile(const WCHAR* szFileName, LPCSTR szEntryPoint, LPCSTR
szShaderModel, ID3DBlob** ppBlobOut)
```

```
{
```

```
    HRESULT hr = S_OK;
```

```
    ID3DBlob* pErrorBlob = NULL;
```

```
    hr = D3DCompileFromFile(szFileName, NULL, NULL, szEntryPoint, szShaderModel,
        D3DCOMPILER_ENABLE_STRICTNESS, 0, ppBlobOut, &pErrorBlob);
```

```
// === ИСПРАВЛЕНИЕ: ТЕПЕРЬ ПЕРЕХВАТ НА СВОЕМ ЗАКОННОМ МЕСТЕ ВНУТРИ ФУНКЦИИ ===
```

```
    if (FAILED(hr))
```

```
    {
```

```
        if (pErrorBlob != NULL)
```

```
        {
```

```
            char* compileErrors = (char*)(pErrorBlob->GetBufferPointer());
```

```

size_t newsize = strlen(compileErrors) + 1;
wchar_t* wcstring = new wchar_t[newsize];
size_t convertedChars = 0;
mbstowcs_s(&convertedChars, wcstring, newsize, compileErrors, _TRUNCATE);

// Показываем на экране точную строчную ошибку из файла shaders.hlsl
MessageBoxW(NULL, wcstring, L"Критическая ошибка компиляции HLSL!", MB_OK |
MB_ICONERROR);

delete[] wcstring;
pErrorBlob->Release();
}
else
{
    MessageBoxW(NULL, L"Файл shaders.hlsl не найден или поврежден!", L"Ошибка", MB_OK |
MB_ICONERROR);
}
return hr;
}

return S_OK;
}

// =====
// НОВАЯ ФУНКЦИЯ ДИНАМИЧЕСКОЙ НАСТРОЙКИ ПОЛЗУНКОВ ПОД ТЕКУЩИЙ РЕЖИМ
// =====
void UpdateDlgScrollbars(HWND hWnd)
{
    if (g_currentImgWidth == 0 || g_currentImgHeight == 0) return;

    SCROLLINFO si = {};
    si.cbSize = sizeof(SCROLLINFO);
    si.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;

```

```

si.nMin = 0;

// Получаем реальные размеры рабочей области отладочного окна прямо сейчас
RECT rcClient;
GetClientRect(hWnd, &rcClient);
int winW = rcClient.right - rcClient.left;
int winH = rcClient.bottom - rcClient.top; // Вычитаем 40 пикселей под меню "Режимы"

if (winW <= 0) winW = 400;
if (winH <= 0) winH = 320;

si.nPage = winW;

if (g_diagMode == 2) // Если выбран Режим 3: Сжатый в 8 раз
{
    UINT compressedW = g_currentImgWidth / 8;
    UINT compressedH = g_currentImgHeight / 8;

    si.nPage = winW;
    si.nMax = compressedW;
    if (g_dlgScrollX_8x8 > (si.nMax - (int)si.nPage)) g_dlgScrollX_8x8 = (si.nMax > (int)si.nPage) ?
(si.nMax - (int)si.nPage) : 0;
    si.nPos = g_dlgScrollX_8x8;
    SetScrollInfo(hWnd, SB_HORZ, &si, TRUE);

    si.nPage = winH; // Размер вертикальной страницы равен физической высоте окна winH
    si.nMax = compressedH;
    if (g_dlgScrollY_8x8 > (si.nMax - (int)si.nPage)) g_dlgScrollY_8x8 = (si.nMax > (int)si.nPage) ?
(si.nMax - (int)si.nPage) : 0;
    si.nPos = g_dlgScrollY_8x8;
    SetScrollInfo(hWnd, SB_VERT, &si, TRUE);
}
else // Для Режимов 1 и 2 (Исходный и Монохромный)

```

```

{
    // Исходные кадры отображаются с укрупнением в 2 раза
    si.nMax = g_currentImgWidth;

    if (g_dlgScrollX > (si.nMax - (int)si.nPage)) g_dlgScrollX = (si.nMax > (int)si.nPage) ? (si.nMax - (int)si.nPage) : 0;

    si.nPos = g_dlgScrollX;

    SetScrollInfo(hWnd, SB_HORZ, &si, TRUE);

    si.nPage = winH;

    si.nMax = g_currentImgHeight;

    if (g_dlgScrollY > (si.nMax - (int)si.nPage)) g_dlgScrollY = (si.nMax > (int)si.nPage) ? (si.nMax - (int)si.nPage) : 0;

    si.nPos = g_dlgScrollY;

    SetScrollInfo(hWnd, SB_VERT, &si, TRUE);
}
}

```

```

void UpdateDiagnosticStatusText(HWND hWnd)
{
    if (!g_hDlgStatusWnd) return;

    wchar_t statusBuf[256];

    if (g_diagMode == 2) // Режим 3: Сжатый в 8 раз
    {
        wprintf(statusBuf, L"Сжатый режим 8x8 | Позиция X: %d, Y: %d",
            g_dlgScrollX_8x8, g_dlgScrollY_8x8);
    }
    else // Режимы 1 и 2: Исходный и Монохромный
    {
        wprintf(statusBuf, L"Режим 1:1 | Позиция X: %d, Y: %d",
            g_dlgScrollX, g_dlgScrollY);
    }
}

```

```

// Отправляем текст в первую секцию строки статуса диагностического окна
SendMessage(g_hDlgStatusWnd, SB_SETTEXT, 0, (LPARAM)statusBuf);
}

// === 1. ОКОННАЯ ПРОЦЕДУРА ДЛЯ ДИАГНОСТИЧЕСКОЙ ПАНЕЛИ ===
LRESULT CALLBACK DiagnosticWndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {
    case WM_HSCROLL:
    {
        SCROLLINFO si = {};
        si.cbSize = sizeof(SCROLLINFO);
        si.fMask = SIF_ALL; // Запрашиваем абсолютно все параметры, включая nTrackPos
        GetScrollInfo(hWnd, SB_HORZ, &si);

        int oldPos = si.nPos;

        // В Win32 при отпускании мыши после перетаскивания приходит код SB_THUMBPOSITION,
        // и реальная позиция лежит строго в si.nTrackPos. Извлекаем её:
        if (LOWORD(wParam) == SB_THUMBPOSITION || LOWORD(wParam) == SB_THUMBTRACK)
        {
            si.nPos = si.nTrackPos;
        }
        else
        {
            // Обработка кликов по стрелочкам и пустому полю ползунка
            switch (LOWORD(wParam))
            {
            case SB_LINELEFT: si.nPos -= 10; break; // Удобный шаг сдвига на 10 пикселей
            case SB_LINERIGHT: si.nPos += 10; break;
            case SB_PAGELEFT: si.nPos -= si.nPage; break;

```

```

        case SB_PAGERIGHT: si.nPos += si.nPage; break;
    }
}

// Жестко удерживаем бегунок в рамках доступного размера кадра
if (si.nPos < si.nMin) si.nPos = si.nMin;
if (si.nPos > (si.nMax - (int)si.nPage)) si.nPos = si.nMax - (int)si.nPage;

if (si.nPos != oldPos)
{
    // ИСПРАВЛЕНИЕ: строго записываем чистую позицию без каких-либо искажений масштаба
    if (g_diagMode == 2) {
        g_dlgScrollX_8x8 = si.nPos;
    }
    else {
        g_dlgScrollX = si.nPos; // Чистый сдвиг 1:1 для Режимов 1 и 2
    }

    si.fMask = SIF_POS;

    SetScrollInfo(hWnd, SB_HORZ, &si, TRUE); // Обновляем позицию бегунка на рамке
    // НОВЫЙ КОД: Обновляем координаты в строке статуса
    UpdateDiagnosticStatusText(hWnd);

    InvalidateRect(hWnd, NULL, FALSE); // Вызываем принудительный WM_PAINT кадра
}

return 0;
}

// Обработка вертикальной прокрутки
case WM_VSCROLL:
{
    SCROLLINFO si = {};

    si.cbSize = sizeof(SCROLLINFO);

```

```

si.fMask = SIF_ALL;

GetScrollInfo(hWnd, SB_VERT, &si);

int oldPos = si.nPos;

if (LOWORD(wParam) == SB_THUMBPOSITION || LOWORD(wParam) == SB_THUMBTRACK)
{
    si.nPos = si.nTrackPos;
}
else
{
    // Обработка кликов по стрелочкам и пустому полю ползунка
    switch (LOWORD(wParam))
    {
        case SB_LINEUP: si.nPos -= 10; break; // Удобный шаг сдвига на 10 пикселей
        case SB_LINEDOWN: si.nPos += 10; break;
        case SB_PAGEUP: si.nPos -= si.nPage; break;
        case SB_PAGEDOWN: si.nPos += si.nPage; break;
    }
}

// Проверяем рамки диапазона
if (si.nPos < si.nMin) si.nPos = si.nMin;
if (si.nPos > (si.nMax - (int)si.nPage)) si.nPos = (si.nMax - (int)si.nPage);

if (si.nPos != oldPos)
{
    if (g_diagMode == 2) {
        g_dlgScrollY_8x8 = si.nPos; // Строго для Режим 3 (case 2) пишем в сжатую координату
    }
    else {
        g_dlgScrollY = si.nPos; // Для Режимов 1 и 2 (case 0 и case 1) пишем в обычную
        координату
    }
}

```

```

    }

    si.fMask = SIF_POS;

    SetScrollInfo(hWnd, SB_VERT, &si, TRUE); // Перерисовываем сам бегунок
    // НОВЫЙ КОД: Обновляем координаты в строке статуса
    UpdateDiagnosticStatusText(hWnd);

    InvalidateRect(hWnd, NULL, FALSE); // Заставляем окно перерисовать картинку
}

return 0;
}

case WM_SIZE:

    return 0; // Блокируем промежуточную перерисовку во время растягивания

case WM_EXITSIZEMOVE:

    // Сначала принудительно обновляем размеры страниц ползунков под новую растянутую
    рамку окна!

    UpdateDlgScrollbars(hWnd);

    // Только после этого перерисовываем кадр в масштабе 1:1
    InvalidateRect(hWnd, NULL, FALSE);

    return 0;

case WM_COMMAND:
{
    int wmlId = LOWORD(wParam);

    // Диапазон пунктов от 2000 (Режим 1) до 2007 (Режим 8)
    if (wmlId >= 2000 && wmlId <= 2007)
    {
        g_diagMode = wmlId - 2000;

        // Получаем доступ к меню панели диагностики
        HMENU hMenuBar = GetMenu(hWnd);

        if (hMenuBar) {

            HMENU hPopupMenu = GetSubMenu(hMenuBar, 0); // Самое первое выпадающее подменю
            "Режимы"

```

```

        if (hPopupMenu) {
            // Снимаем галочки со всех восьми режимов панели
            for (int id = 2000; id <= 2007; id++) {
                CheckMenuItem(hPopupMenu, id, MF_BYCOMMAND | MF_UNCHECKED);
            }
            // Ставим галочку на выбранный режим
            CheckMenuItem(hPopupMenu, wParam, MF_BYCOMMAND | MF_CHECKED);
        }
    }

    wchar_t titleBuf[128];
    wprintf(titleBuf, L"HDR Дефектоскоп — Активен Режим %d", g_diagMode + 1);
    SetWindowText(hWnd, titleBuf);

    UpdateDlgScrollbars(hWnd); // Перестраиваем диапазоны ползунков под выбранный режим
    кадра

```

// УДАЛИЛИ ПРИНУДИТЕЛЬНЫЙ ТЯЖЕЛЫЙ ФЛАГ GPU, ЧТОБЫ УБРАТЬ 10-СЕКУНДНОЕ ЗАВИСАНИЕ ОКНА!

```

        InvalidateRect(hWnd, NULL, FALSE);
        return 0;
    }
}

break;

```

```

case WM_PAINT:

```

```

{
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);

```

```

    // Берем только ту область, которую Windows официально просит обновить (ps.rcPaint)

```

```

    // Это предотвратит повторные циклы рисования поверх уже готового кадра

```

```

    HBRUSH hBrush = NULL;

```

```

switch (g_diagMode)
{
case 0: // Режим 1: Исходный
{
    // Очищаем фон ОДИН раз
    HBRUSH hBg = CreateSolidBrush(RGB(0, 0, 0));
    FillRect(hdc, &ps.rcPaint, hBg); // Очищаем строго область запроса, а не весь 400x400
    DeleteObject(hBg);

    if (pPixelsBuffer != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
    {
        // Прямо запрашиваем у Windows текущие размеры видимой страницы ползунков
        SCROLLINFO siH = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
        GetScrollInfo(hWnd, SB_HORZ, &siH);
        SCROLLINFO siV = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
        GetScrollInfo(hWnd, SB_VERT, &siV);

        // В режиме 1:1 индексы считывания равны чистой позиции бегунка
        UINT startX = siH.nPos;
        UINT startY = siV.nPos;

        // Граница перебора равна позиции старта плюс физическая ширина видимой страницы
        UINT endX = startX + siH.nPage;
        UINT endY = startY + siV.nPage;

        if (endX > g_currentImgWidth) endX = g_currentImgWidth;
        if (endY > g_currentImgHeight) endY = g_currentImgHeight;

        for (UINT y = startY; y < endY; y++)
        {
            UINT rowOffset = y * g_currentImgWidth;
            for (UINT x = startX; x < endX; x++)

```

окна

```

    {
        DWORD color = pPixelsBuffer[rowOffset + x];
        BYTE b = (BYTE)(color & 0xFF);
        BYTE g = (BYTE)((color >> 8) & 0xFF);
        BYTE r = (BYTE)((color >> 16) & 0xFF);

        HBRUSH hPxlBrush = CreateSolidBrush(RGB(r, g, b));
        RECT pxlRect = {
            (int)(x - startX),
            (int)(y - startY),
            (int)(x - startX + 1),
            (int)(y - startY + 1)
        };
        FillRect(hdc, &pxlRect, hPxlBrush);
        DeleteObject(hPxlBrush);
    }
}

break;
case 1: // Режим 2: Монохромный
{
    HBRUSH hBg = CreateSolidBrush(RGB(0, 0, 0));
    FillRect(hdc, &ps.rcPaint, hBg);
    DeleteObject(hBg);

    if (pPixelsBuffer != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
    {
        // Прямо запрашиваем у Windows текущие размеры видимой страницы ползунков
        SCROLLINFO siH = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
        GetScrollInfo(hWnd, SB_HORZ, &siH);
        SCROLLINFO siV = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
    }
}

```

```

GetScrollInfo(hWnd, SB_VERT, &siV);

// В режиме 1:1 индексы считывания равны чистой позиции бегунка
UINT startX = siH.nPos;
UINT startY = siV.nPos;

// Граница перебора равна позиции старта плюс физическая ширина видимой страницы
окна
UINT endX = startX + siH.nPage;
UINT endY = startY + siV.nPage;

if (endX > g_currentImgWidth) endX = g_currentImgWidth;
if (endY > g_currentImgHeight) endY = g_currentImgHeight;

for (UINT y = startY; y < endY; y++)
{
    UINT rowOffset = y * g_currentImgWidth;
    for (UINT x = startX; x < endX; x++)
    {
        DWORD color = pPixelsBuffer[rowOffset + x];
        BYTE b = (BYTE)(color & 0xFF);
        BYTE g = (BYTE)((color >> 8) & 0xFF);
        BYTE r = (BYTE)((color >> 16) & 0xFF);

        float grayVal = (float)r * 0.299f + (float)g * 0.587f + (float)b * 0.114f;
        BYTE gray = (BYTE)grayVal;

        HBRUSH hPxlBrush = CreateSolidBrush(RGB(gray, gray, gray));
        RECT pxlRect = {
            (int)(x - startX),
            (int)(y - startY),
            (int)(x - startX + 1),
            (int)(y - startY + 1)
        }
    }
}

```

```

        };

        FillRect(hdc, &pxlRect, hPxlBrush);

        DeleteObject(hPxlBrush);

    }

}

}

break;

case 2: // Режим 3: Сжатый в 8 раз (ВЫВОД БЛОКА ПАМЯТИ ЦЕЛИКОМ ЗА 0 СЕКУНД)
{
    if (pCompressedBuffer8x != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
    {
        UINT compressedW = g_currentImgWidth / 8;
        UINT compressedH = g_currentImgHeight / 8;

        UINT startX = (UINT)g_dlgScrollX_8x8;
        UINT startY = (UINT)g_dlgScrollY_8x8;

        RECT rcClient;
        GetClientRect(hWnd, &rcClient);
        int winW = rcClient.right - rcClient.left;
        int winH = rcClient.bottom - rcClient.top;

        if (winW <= 0 || winH <= 0) break;

        // Описываем для Windows структуру нашего блока памяти в ОЗУ (DIB)
        BITMAPINFO bmi = {};
        bmi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
        bmi.bmiHeader.biWidth = compressedW;

        // Отрицательная высота biHeight заставляет Windows читать массив сверху вниз (как у нас
        в ОЗУ)
        bmi.bmiHeader.biHeight = -(int)compressedH;
        bmi.bmiHeader.biPlanes = 1;
    }
}

```

```

bmi.bmiHeader.biBitCount = 32; // Честные 32 бита (DWORD на пиксель)

bmi.bmiHeader.biCompression = BI_RGB;

// ИСПРАВЛЕНИЕ: Вычисляем реальный физический остаток пикселей в буфере памяти
ОЗУ,

// который мы можем безопасно прочитать без переполнения WinAPI!
int viewW = ((startX + winW) > compressedW) ? (int)(compressedW - startX) : winW;
int viewH = ((startY + winH) > compressedH) ? (int)(compressedH - startY) : winH;

// Очищаем фон окна от старых остатков
HBRUSH hBg = CreateSolidBrush(RGB(0, 0, 0));
FillRect(hdc, &rcClient, hBg);
DeleteObject(hBg);

// Передаем блок данных. Высота источника и назначения теперь синхронизированы
// с доступным остатком памяти кадра viewH!
StretchDIBits(
    hdc,
    0, 0, viewW, viewH,      // Куда выводим на экране панели (левый верхний угол)
    (int)startX, (int)startY, // Откуда начинаем считывание из массива в ОЗУ
    viewW, viewH,          // Сколько пикселей считываем по осям X и Y
    pCompressedBuffer8x,    // Указатель на массив пикселей кадра
    &bmi,
    DIB_RGB_COLORS,
    SRCCOPY
);
}
}

break;

case 3: hBrush = CreateSolidBrush(RGB(0, 120, 30)); // Режим 3 - Зеленый
    break;

case 4: hBrush = CreateSolidBrush(RGB(0, 50, 150)); // Режим 4 - Синий
    break;

```

```

    }

    /*
    if (hBrush)
    {
        // Описываем полезную квадратную область 400x400 под меню
        RECT rect = { 0, 0, 400, 400 };
        // Процессор закрашивает окно выбранным цветом
        FillRect(hdc, &rect, hBrush);
        DeleteObject(hBrush);
    }
    */

    // Процессор закрывает рисование
    EndPaint(hWnd, &ps);
    return 0;
}

break;

case WM_CLOSE:
    ShowWindow(hWnd, SW_HIDE);
    return 0;
case WM_DESTROY:
    return 0;
default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}

return 0;
}

// === 2. САМА ФУНКЦИЯ СОЗДАНИЯ ДИАГНОСТИЧЕСКОГО ОКНА С МЕНЮ ===
void CreateDiagnosticWindow(HINSTANCE hInstance, HWND hParentWnd)
{

```

```

WNDCLASSEX wcex = {};

wcex.cbSize = sizeof(WNDCLASSEX);

wcex.style = CS_HREDRAW | CS_VREDRAW;

wcex.lpfnWndProc = DiagnosticWndProc;

wcex.hInstance = hInstance;

wcex.hCursor = LoadCursor(NULL, IDC_ARROW);

wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

wcex.lpszClassName = L"HDR_Diagnostic_Class";


RegisterClassEx(&wcex);


HMENU hMenuBar = CreateMenu();

HMENU hPopupMenu = CreatePopupMenu();


if (hPopupMenu)
{
    AppendMenu(hPopupMenu, MF_STRING, 2000, L"Режим 1: Исходный");
    AppendMenu(hPopupMenu, MF_STRING, 2001, L"Режим 2: Монохромный");
    AppendMenu(hPopupMenu, MF_STRING, 2002, L"Режим 3: Сжатый в 8 раз");
    AppendMenu(hPopupMenu, MF_STRING, 2003, L"Режим 4: Растянутый после сжатия");
    AppendMenu(hPopupMenu, MF_STRING, 2004, L"Режим 5: Разница монохромного и
растянутого");
    AppendMenu(hPopupMenu, MF_STRING, 2005, L"Режим 6: Сжатый и применена гамма-
коррекция");
    AppendMenu(hPopupMenu, MF_STRING, 2006, L"Режим 7: Растянутый после гамма-
коррекции");
    AppendMenu(hPopupMenu, MF_STRING, 2007, L"Режим 8: Сумма той разности и растянутого
после гамма-коррекции");
}

AppendMenu(hMenuBar, MF_POPUP, (UINT_PTR)hPopupMenu, L"Режимы");


RECT parentRect;

GetWindowRect(hParentWnd, &parentRect);

int posX = parentRect.right + 10;

```

```

int posY = parentRect.top;

g_hDlgWnd = CreateWindowEx(
    0, L"HDR_Diagnostic_Class", L"HDR Дефектоскоп — Панель Диагностики",
    WS_OVERLAPPEDWINDOW | WS_CLIPCHILDREN | WS_HSCROLL | WS_VSCROLL,
    posX, posY, 400, 400,
    hParentWnd, hMenuBar, hInstance, NULL
);

if (!g_hDlgWnd) return;

g_hDlgStatusWnd = CreateStatusWindow(
    WS_CHILD | WS_VISIBLE,          // Стандартные стили из нашего эталона 2004 года!
    L"Диагностика активна. Ожидание кадра.", // Стартовый текст для правого окна
    g_hDlgWnd,                      // Родительское окно (наша панель диагностики)
    2006                           // Уникальный ID второго статус-бара
);

ShowWindow(g_hDlgWnd, SW_SHOW);

if (g_hDlgWnd)
{
    HMENU hDlgMenu = GetMenu(g_hDlgWnd);
    if (hDlgMenu)
    {
        HMENU hModesMenu = GetSubMenu(hDlgMenu, 0); // Самое первое выпадающее меню
        "Режимы"
        if (hModesMenu)
        {
            // Ставим галочку на "Режим 1: Исходный" сразу при создании окна
            CheckMenuItem(hModesMenu, 2000, MF_BYCOMMAND | MF_CHECKED);
        }
    }
}

```

```

    }

    UpdateWindow(g_hDlgWnd);
}

// Инициализация видеокарты и создание объектов
HRESULT InitDevice(HWND hwnd)
{
    HRESULT hr = S_OK;

    RECT rc;
    GetClientRect(hwnd, &rc);
    UINT width = rc.right - rc.left;
    UINT height = rc.bottom - rc.top;

    DXGI_SWAP_CHAIN_DESC sd = {};
    sd.BufferCount = 1;
    sd.BufferDesc.Width = width;
    sd.BufferDesc.Height = height;
    sd.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    sd.BufferDesc.RefreshRate.Numerator = 60;
    sd.BufferDesc.RefreshRate.Denominator = 1;
    sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
    sd.OutputWindow = hwnd;
    sd.SampleDesc.Count = 1;
    sd.SampleDesc.Quality = 0;
    sd.Windowed = TRUE;

    D3D_DRIVER_TYPE driverType = D3D_DRIVER_TYPE_HARDWARE;
    D3D_FEATURE_LEVEL featureLevel = D3D_FEATURE_LEVEL_11_0;

    hr = D3D11CreateDeviceAndSwapChain(NULL, driverType, NULL, 0, &featureLevel, 1,
        D3D11_SDK_VERSION, &sd, &g_pSwapChain, &g_pd3dDevice, NULL, &g_pImmediateContext);
}

```

```
if (FAILED(hr)) return hr;
```

```
ID3D11Texture2D* pBackBuffer = NULL;
```

```
hr = g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);
```

```
if (FAILED(hr)) return hr;
```

```
hr = g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
```

```
pBackBuffer->Release();
```

```
if (FAILED(hr)) return hr;
```

```
g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);
```

```
D3D11_VIEWPORT vp;
```

```
vp.Width = (FLOAT)width;
```

```
vp.Height = (FLOAT)height;
```

```
vp.MinDepth = 0.0f;
```

```
vp.MaxDepth = 1.0f;
```

```
vp.TopLeftX = 0;
```

```
vp.TopLeftY = 0;
```

```
g_pImmediateContext->RSSetViewports(1, &vp);
```

```
// Компиляция Вершинного Шейдера
```

```
ID3DBlob* pVSBlob = NULL;
```

```
hr = CompileShaderFromFile(L"shaders.hlsl", "VS", "vs_4_0", &pVSBlob);
```

```
// === ИСПРАВЛЕНИЕ: ПЕРЕХВАТ ТЕКСТА ОШИБКИ ИЗ SHADERS.HLSL ===
```

```
if (FAILED(hr)) return hr; // Вернули стандартную проверку, ошибка C2065 исчезнет!
```

```
hr = g_pd3dDevice->CreateVertexShader(pVSBlob->GetBufferPointer(), pVSBlob->GetBufferSize(),  
NULL, &g_pVertexShader);
```

```
if (FAILED(hr)) {
```

```

        MessageBoxW(hwnd, L"Ошибка на Stage Ш-VS: Видеокарта отвергла Вершинный Шейдер!",
L"Сбой дефектоскопа", MB_OK | MB_ICONERROR);

        pVSBlob->Release();

        return hr;
    }

    // Описываем формат вершин для видеокарты (Input Layout)
    D3D11_INPUT_ELEMENT_DESC layout[] =
    {
        { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
        { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0 },
    };

    UINT numElements = sizeof(layout) / sizeof(layout[0]);

    hr = g_pd3dDevice->CreateInputLayout(layout, numElements, pVSBlob->GetBufferPointer(), pVSBlob-
>GetBufferSize(), &g_pVertexLayout);

    pVSBlob->Release();

    if (FAILED(hr)) return hr;

    g_pImmediateContext->IASetInputLayout(g_pVertexLayout);

    // Компиляция Пиксельного Шейдера
    ID3DBlob* pPSBlob = NULL;

    hr = CompileShaderFromFile(L"shaders.hlsl", "PS", "ps_4_0", &pPSBlob);

    if (FAILED(hr)) return hr;

    hr = g_pd3dDevice->CreatePixelShader(pPSBlob->GetBufferPointer(), pPSBlob->GetBufferSize(), NULL,
&g_pPixelShader);

    pPSBlob->Release();

    if (FAILED(hr)) return hr;

    // КОМПИЛЯЦИЯ И СОЗДАНИЕ НАШЕГО ШЕЙДЕРА КОПИРОВАНИЯ
    ID3DBlob* pPSCopyBlob = NULL;

```

```

hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Copy", "ps_4_0", &pPSCopyBlob);

if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreatePixelShader(pPSCopyBlob->GetBufferPointer(), pPSCopyBlob-
>GetBufferSize(), NULL, &g_pPixelShaderCopy);

pPSCopyBlob->Release();

if (FAILED(hr)) return hr;


// === КОМПИЛЯЦИЯ НОВОГО ШЕЙДЕРА ПИРАМИДАЛЬНОГО СЖАТИЯ ===

ID3DBlob* pDownsampleBlob = NULL; // ОБЪЯВЛЯЕМ ПРОПУЩЕННЫЙ БУФЕР ДЛЯ СЖАТИЯ

hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Downsample2X", "ps_4_0", &pDownsampleBlob);

if (FAILED(hr)) return hr;


hr = g_pd3dDevice->CreatePixelShader(pDownsampleBlob->GetBufferPointer(), pDownsampleBlob-
>GetBufferSize(), NULL, &g_pPixelShaderDownsample);

pDownsampleBlob->Release(); // Чисто освобождаем буфер

if (FAILED(hr)) return hr;


// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 4-ГО ШЕЙДЕРА (STAGE 2) ===

ID3DBlob* pPSStage2Blob = NULL;

// Указываем точку входа "PS_Stage2" — строго как имя функции в файле shaders.hlsl

hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage2", "ps_4_0", &pPSStage2Blob);

if (FAILED(hr)) return hr;


hr = g_pd3dDevice->CreatePixelShader(pPSStage2Blob->GetBufferPointer(), pPSStage2Blob-
>GetBufferSize(), NULL, &g_pPixelShaderStage2);

pPSStage2Blob->Release(); // Освобождаем память временного буфера компиляции

if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===


// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 5-ГО ШЕЙДЕРА (STAGE 3) ===

ID3DBlob* pPSStage3Blob = NULL;

// Указываем точку входа "PS_Stage3" — строго как имя функции в файле shaders.hlsl

hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage3", "ps_4_0", &pPSStage3Blob);

```

```
if (FAILED(hr)) return hr;
```

```
hr = g_pd3dDevice->CreatePixelShader(pPSStage3Blob->GetBufferPointer(), pPSStage3Blob->GetBufferSize(), NULL, &g_pPixelShaderStage3);
```

```
pPSStage3Blob->Release(); // Освобождаем память временного буфера компиляции
```

```
if (FAILED(hr)) return hr;
```

```
// === КОНЕЦ НОВОГО КОДА ===
```

```
// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 6-ГО ШЕЙДЕРА (FINAL) ===
```

```
ID3DBlob* pPSFinalBlob = NULL;
```

```
// Указываем точку входа "PS_Final" — строго как имя функции в файле shaders.hlsl
```

```
hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Final", "ps_4_0", &pPSFinalBlob);
```

```
if (FAILED(hr)) return hr;
```

```
hr = g_pd3dDevice->CreatePixelShader(pPSFinalBlob->GetBufferPointer(), pPSFinalBlob->GetBufferSize(), NULL, &g_pPixelShaderFinal);
```

```
pPSFinalBlob->Release(); // Освобождаем память временного буфера компиляции
```

```
if (FAILED(hr)) return hr;
```

```
// === КОНЕЦ НОВОГО КОДА ===
```

```
// Создаем тестовую текстуру 2x2 пикселя (Черный и Белый в шахматном порядке)
```

```
UINT textureData[] = {
```

```
    0xFFFFFFFF, 0xFF000000,
```

```
    0xFF000000, 0xFFFFFFFF
```

```
};
```

```
D3D11_TEXTURE2D_DESC desc = {};
```

```
desc.Width = 2;
```

```
desc.Height = 2;
```

```
desc.MipLevels = 1;
```

```
desc.ArraySize = 1;
```

```
desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
```

```
desc.SampleDesc.Count = 1;
```

```

desc.Usage = D3D11_USAGE_DEFAULT;

desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;


D3D11_SUBRESOURCE_DATA initData = {};

initData.pSysMem = textureData;

initData.SysMemPitch = 2 * sizeof(UINT);


ID3D11Texture2D* pTexture = NULL;

hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);

if (SUCCEEDED(hr))
{
    // Описываем параметры уменьшенного холста (46x46 для кадра 184)
    D3D11_TEXTURE2D_DESC macroDesc = {};

    macroDesc.Width = 46; // Ровно в 4 раза меньше базовой ширины кадра
    macroDesc.Height = 46; // Ровно в 4 раза меньше базовой высоты кадра
    macroDesc.MipLevels = 1;
    macroDesc.ArraySize = 1;
    macroDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Стандартный формат RGBA
    macroDesc.SampleDesc.Count = 1;
    macroDesc.Usage = D3D11_USAGE_DEFAULT;


    // КРИТИЧЕСКИЙ ФЛАГ: Разрешаем видеокарте и записывать туда (RTV), и читать из нее (SRV)
    macroDesc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;


    // 1. Физически создаем холст макротекстуры в видеопамяти GPU
    hr = g_pd3dDevice->CreateTexture2D(&macroDesc, NULL, &g_pMacroTexture4x4);


    if (SUCCEEDED(hr))
    {
        hr = g_pd3dDevice->CreateRenderTargetView(g_pMacroTexture4x4, NULL, &g_pMacroRTV4x4);
        if (SUCCEEDED(hr))
        {

```

```

        hr = g_pd3dDevice->CreateShaderResourceView(g_pMacroTexture4x4, NULL,
&g_pMacroSRV4x4);
    }
}

D3D11_TEXTURE2D_DESC stagingDesc = {};
stagingDesc.Width = 46; // Размер строго совпадает с нашей матрицей макроблоков
stagingDesc.Height = 46;
stagingDesc.MipLevels = 1;
stagingDesc.ArraySize = 1;
stagingDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Стандартный формат RGBA кадра
stagingDesc.SampleDesc.Count = 1;
stagingDesc.SampleDesc.Quality = 0;

// Настраиваем специальный режим передачи данных из видеокарты в процессор:
stagingDesc.Usage = D3D11_USAGE_STAGING; // Режим инспекционного буфера
stagingDesc.BindFlags = 0; // К шейдерам этот буфер не привязывается
stagingDesc.CPUAccessFlags = D3D11_CPU_ACCESS_READ; // РАЗРЕШАЕМ ПРОЦЕССОРУ ЧИТАТЬ
ЭТИ БАЙТЫ!

// Физически создаем текстуру-шпион в памяти GPU
hr = g_pd3dDevice->CreateTexture2D(&stagingDesc, NULL, &g_pMacroStaging);
}

if (FAILED(hr)) return hr;

// Настраиваем режим POINT-фильтрации (строго без размытия)
D3D11_SAMPLER_DESC sampDesc = {};
sampDesc.Filter = D3D11_FILTER_MIN_MAG_MIP_POINT;
sampDesc.AddressU = D3D11_TEXTURE_ADDRESS_CLAMP;
sampDesc.AddressV = D3D11_TEXTURE_ADDRESS_CLAMP;
sampDesc.AddressW = D3D11_TEXTURE_ADDRESS_CLAMP;
sampDesc.ComparisonFunc = D3D11_COMPARISON_NEVER;
sampDesc.MinLOD = 0;

```

```

sampDesc.MaxLOD = D3D11_FLOAT32_MAX;

hr = g_pd3dDevice->CreateSamplerState(&sampDesc, &g_pSamplerState);
if (FAILED(hr)) return hr;

// === НОВЫЙ КОД: СОЗДАЕМ БУФЕР КОНСТАНТ НА ВИДЕОКАРТЕ ===
D3D11_BUFFER_DESC cbd = {};
cbd.Usage = D3D11_USAGE_DEFAULT;
cbd.ByteWidth = sizeof(ShaderConstants);    // Размер нашей структуры параметров
cbd.BindFlags = D3D11_BIND_CONSTANT_BUFFER; // Указываем видеокарте, что это буфер
констант
cbd.CPUAccessFlags = 0;

hr = g_pd3dDevice->CreateBuffer(&cbd, NULL, &g_pConstantBuffer);
if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===

// === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 3-ГО ШЕЙДЕРА ===
D3D11_TEXTURE2D_DESC stage1Desc = {};
stage1Desc.Width = 2;                // Размер точно соответствует исходному кадру
stage1Desc.Height = 2;
stage1Desc.MipLevels = 1;
stage1Desc.ArraySize = 1;
stage1Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Стандартный RGBA формат для
хранения данных
stage1Desc.SampleDesc.Count = 1;
stage1Desc.Usage = D3D11_USAGE_DEFAULT;

// Очень важные флаги: текстура будет одновременно холстом для записи и ресурсом для
чтения!
stage1Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pStage1Tex = NULL;
hr = g_pd3dDevice->CreateTexture2D(&stage1Desc, NULL, &pStage1Tex);

```

```

if (SUCCEEDED(hr))
{
    // Создаем "взгляд для записи" (RenderTargetView) — сюда 3-й шейдер будет писать данные
    g_pd3dDevice->CreateRenderTargetView(pStage1Tex, NULL, &g_pStage1RTV);

    // Создаем "взгляд для чтения" (ShaderResourceView) — отсюда 4-й шейдер будет их забирать
    g_pd3dDevice->CreateShaderResourceView(pStage1Tex, NULL, &g_pStage1SRV);

    pStage1Tex->Release(); // Саму текстуру видеокарта удержит внутри интерфейсов
}
if (FAILED(hr)) return hr;
// === КОНЕЦ НОВОГО КОДА ===

// === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 4-ГО ШЕЙДЕРА ===
D3D11_TEXTURE2D_DESC stage2Desc = {};
stage2Desc.Width = 2;           // Размер по-прежнему соответствует тестовому кадру
stage2Desc.Height = 2;
stage2Desc.MipLevels = 1;
stage2Desc.ArraySize = 1;
stage2Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Формат RGBA для хранения
макростатистики
stage2Desc.SampleDesc.Count = 1;
stage2Desc.Usage = D3D11_USAGE_DEFAULT;
stage2Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pStage2Tex = NULL;
hr = g_pd3dDevice->CreateTexture2D(&stage2Desc, NULL, &pStage2Tex);
if (SUCCEEDED(hr))
{
    // Создаем RenderTargetView — сюда 4-й шейдер запишет укрупненные данные 8x8
    g_pd3dDevice->CreateRenderTargetView(pStage2Tex, NULL, &g_pStage2RTV);

    // Создаем ShaderResourceView — отсюда данные заберет финальный сборочный шейдер

```

```

g_pd3dDevice->CreateShaderResourceView(pStage2Tex, NULL, &g_pStage2SRV);

pStage2Tex->Release();
}
if (FAILED(hr)) return hr;
// === КОНЕЦ НОВОГО КОДА ===

ID3D11Texture2D* pStage3Tex = NULL; // НОВОЕ: Объявляем указатель для третьей текстуры
// СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 5-ГО ШЕЙДЕРА
D3D11_TEXTURE2D_DESC stage3Desc = stage2Desc; // Копируем настройки формата и размеров
hr = g_pd3dDevice->CreateTexture2D(&stage3Desc, NULL, &pStage3Tex);

if (SUCCEEDED(hr))
{
    // Создаем RenderTargetView — сюда 5-й шейдер запишет результат фильтрации контраста
    hr = g_pd3dDevice->CreateRenderTargetView(pStage3Tex, NULL, &g_pStage3RTV);

    // Создаем ShaderResourceView — отсюда данные пойдут на финальный экран
    if (SUCCEEDED(hr))
    {
        hr = g_pd3dDevice->CreateShaderResourceView(pStage3Tex, NULL, &g_pStage3SRV);
    }

    pStage3Tex->Release(); // Передаем управление текстурой видеокарте
}
if (FAILED(hr)) return hr;

// Геометрия прямоугольного экрана по умолчанию (стартовая)
SimpleVertex vertices[] =
{
    { { -1.0f, 1.0f, 0.0f }, { 0.0f, 0.0f } },
    { { 1.0f, 1.0f, 0.0f }, { 1.0f, 0.0f } },

```

```
{ { -1.0f, -1.0f, 0.0f }, { 0.0f, 1.0f } },  
{ { 1.0f, -1.0f, 0.0f }, { 1.0f, 1.0f } },  
};
```

```
D3D11_BUFFER_DESC bd = {};  
bd.Usage = D3D11_USAGE_DEFAULT;  
bd.ByteWidth = sizeof(SimpleVertex) * 4;  
bd.BindFlags = D3D11_BIND_VERTEX_BUFFER;  
bd.CPUAccessFlags = 0;
```

```
D3D11_SUBRESOURCE_DATA InitData = {};  
InitData.pSysMem = vertices;
```

```
hr = g_pd3dDevice->CreateBuffer(&bd, &InitData, &g_pVertexBuffer);  
if (FAILED(hr)) return hr;
```

```
// === НАЧАЛО АППАРАТНОЙ ИНИЦИАЛИЗАЦИИ ДИАГНОСТИЧЕСКОГО SWAPCHAIN ===
```

```
if (g_hDlgWnd != NULL && g_pd3dDevice != NULL)
```

```
{
```

```
    // 1. Описываем параметры вывода для второго окна
```

```
    DXGI_SWAP_CHAIN_DESC sdDlg = {};
```

```
    sdDlg.BufferCount = 1;
```

```
    sdDlg.BufferDesc.Width = 400; // Ширина нашего диагностического окна
```

```
    sdDlg.BufferDesc.Height = 400; // Высота нашего диагностического окна
```

```
    sdDlg.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
```

```
    sdDlg.BufferDesc.RefreshRate.Numerator = 60;
```

```
    sdDlg.BufferDesc.RefreshRate.Denominator = 1;
```

```
    sdDlg.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
```

```
    sdDlg.OutputWindow = g_hDlgWnd; // НАПРАВЛЯЕМ ВЫВОД СТРОГО ВО ВТОРОЕ ОКНО!
```

```
    sdDlg.SampleDesc.Count = 1;
```

```
    sdDlg.SampleDesc.Quality = 0;
```

```
    sdDlg.Windowed = TRUE;
```

```

// Извлекаем фабрику DXGI из существующего устройства, чтобы создать SwapChain
IDXGIDevice* pDXGIDevice = NULL;
g_pd3dDevice->QueryInterface(__uuidof(IDXGIDevice), (void**)&pDXGIDevice);

IDXGIAdapter* pDXGIAdapter = NULL;
if (pDXGIDevice) pDXGIDevice->GetParent(__uuidof(IDXGIAdapter), (void**)&pDXGIAdapter);

IDXGIFactory* pIDXGIFactory = NULL;
if (pDXGIAdapter) pDXGIAdapter->GetParent(__uuidof(IDXGIFactory), (void**)&pIDXGIFactory);

if (pIDXGIFactory)
{
    // Создаем цепочку буферов для второго окна
    hr = pIDXGIFactory->CreateSwapChain(g_pd3dDevice, &sdDlg, &g_pDlgsSwapChain);
}

// Освобождаем временные интерфейсы DXGI
if (pIDXGIFactory) pIDXGIFactory->Release();
if (pDXGIAdapter) pDXGIAdapter->Release();
if (pDXGIDevice) pDXGIDevice->Release();

// 2. Создаем цель отрисовки (RenderTargetView) для диагностического окна
if (SUCCEEDED(hr) && g_pDlgsSwapChain)
{
    ID3D11Texture2D* pBackBufferDlg = NULL;
    hr = g_pDlgsSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (void**)&pBackBufferDlg);

    if (SUCCEEDED(hr) && pBackBufferDlg)
    {
        hr = g_pd3dDevice->CreateRenderTargetView(pBackBufferDlg, NULL,
&g_pDlgsRenderTargetView);
        pBackBufferDlg->Release();
    }
}

```

```

    }
}
}

// === КОНЕЦ АППАРАТНОЙ ИНИЦИАЛИЗАЦИИ ===

// === АППАРАТНАЯ ИНИЦИАЛИЗАЦИИ ДИАГНОСТИЧЕСКОГО SWAPCHAIN ===
if (g_hDlgWnd != NULL && g_pd3dDevice != NULL)
{
    DXGI_SWAP_CHAIN_DESC sdDlg = {};
    sdDlg.BufferCount = 1;
    sdDlg.BufferDesc.Width = 400;
    sdDlg.BufferDesc.Height = 400;
    sdDlg.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    sdDlg.BufferDesc.RefreshRate.Numerator = 60;
    sdDlg.BufferDesc.RefreshRate.Denominator = 1;
    sdDlg.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
    sdDlg.OutputWindow = g_hDlgWnd; // Вывод идет строго во второе окно
    sdDlg.SampleDesc.Count = 1;
    sdDlg.SampleDesc.Quality = 0;
    sdDlg.Windowed = TRUE;
}
return S_OK;
}

// ЛЁГКАЯ БЕЗОПАСНАЯ ЗАГЛУШКА, КОТОРАЯ ВЫПОЛНЯЕТСЯ ЗА 0 МИЛЛИСЕКУНД И НЕ ТРОГАЕТ GPU
void Render()
{
    // Просто выходим, ничего не вычисляя на видеокарте во время отладки CPU-режимов
    return;
}

// Обновленная функция отрисовки (Двухпроходный конвейер рендеринга)

```

```

// Обновленная функция отрисовки (Трехпроходный вычислительный конвейер)
// Обновленная функция отрисовки (Четырехпроходный вычислительный конвейер)
// Финальная функция отрисовки (Пятипроходный вычислительный конвейер)
// ПОЛНОЦЕННАЯ синхронизированная функция отрисовки (ПЯТИПРОХОДНЫЙ КОНВЕЙЕР)
// ПРЯМОЙ СКВОЗНОЙ КОНВЕЙЕР: ИСКЛЮЧАЕТ ЛЮБЫЕ БЛОКИРОВКИ ТЕКСТУР В ПАМЯТИ
// ПОЛНОЦЕННАЯ синхронизированная функция отрисовки с автоматическим
СКРОЛЛИНГОМ И СОХРАНЕНИЕМ ПРОПОРЦИЙ
// УТРЕННЯЯ ИСПРАВЛЕННАЯ функция отрисовки с умным портом просмотра
// ТОЧНАЯ КОПИЯ УТРЕННЕЙ РАБОЧЕЙ функции отрисовки (МЯГКИЕ ПОЛУТОНА И СЕРОЕ ПОЛЕ)
/*
void Render()
{
    if (g_pRenderTargetView == NULL || g_pTextureSRV == NULL) return;

    RECT rcClient;
    if (hwnd && GetClientRect(hwnd, &rcClient))
    {
        g_wndW = (float)(rcClient.right - rcClient.left);
        g_wndH = (float)(rcClient.bottom - rcClient.top);
    }

    // Защита от нулевых размеров (если окно свёрнуто)
    if (g_wndW <= 0.0f || g_wndH <= 0.0f) { g_wndW = 800.0f; g_wndH = 600.0f; }

    // ОБЯЗАТЕЛЬНО: Настройка общих параметров сетки вершин (вызывается всегда)
    UINT stride = sizeof(SimpleVertex);
    UINT offset = 0;
    g_pImmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);
    g_pImmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);
    g_pImmediateContext->IASetInputLayout(g_pVertexLayout);
    g_pImmediateContext->VSSetShader(g_pVertexShader, NULL, 0);

    ID3D11ShaderResourceView* nullSRV[] = { NULL };

```

```

ID3D11RenderTargetView* nullRTV[] = { NULL };

float ClearColorBlack[] = { 0.0f, 0.0f, 0.0f, 1.0f };

UINT fileWidth = g_currentImgWidth;
UINT fileHeight = g_currentImgHeight;
float renderW = (float)fileWidth;
float renderH = (float)fileHeight;

// =====
// ЗОНА ТЯЖЕЛЫХ ВЫЧИСЛЕНИЙ: ТОЛЬКО ДЛЯ ОДНОКРАТНОЙ ФИЛЬТРАЦИИ ПРИ
ОТКРЫТИИ/СМЕНЕ РЕЖИМА
// =====

if (g_bNeedsUpdate)
{
    if (g_pStage3RTV && g_pStageMasterRTV)
    {
        if (g_hDlgStatusWnd) SendMessage(g_hDlgStatusWnd, SB_SETTEXT, 0, (LPARAM)L"GPU Пасс A:
Фильтрация...");

        D3D11_VIEWPORT vpS3 = { 0.0f, 0.0f, renderW, renderH, 0.0f, 1.0f };

        g_pImmediateContext->RSSetViewports(1, &vpS3);

        ShaderConstants cbDataLocal = {};
        cbDataLocal.width = renderW;
        cbDataLocal.height = renderH;
        cbDataLocal.d_width = 1.0f / renderW;
        cbDataLocal.d_height = 1.0f / renderH;
        cbDataLocal.splitX = 0.5f;

        g_pImmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbDataLocal, 0, 0);

        g_pImmediateContext->ClearRenderTargetView(g_pStage3RTV, ClearColorBlack);
        g_pImmediateContext->OMSetRenderTargets(1, &g_pStage3RTV, NULL);
        g_pImmediateContext->PSSetShader(g_pPixelShaderStage3, NULL, 0);
    }
}

```

```
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);  
g_pImmediateContext->Draw(4, 0);  
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
```

```
if (g_pMacroStaging) {  
    ID3D11Resource* pStage3Res = NULL;  
    g_pStage3RTV->GetResource(&pStage3Res);  
    if (pStage3Res) {  
        g_pImmediateContext->CopyResource(g_pMacroStaging, pStage3Res);  
        pStage3Res->Release();  
    }  
}
```

```
if (g_hDlgStatusWnd) SendMessage(g_hDlgStatusWnd, SB_SETTEXT, 0, (LPARAM)L"GPU Пасс Б:  
Сборка мастер-кадра...");
```

```
g_pImmediateContext->ClearRenderTargetView(g_pStageMasterRTV, ClearColorBlack);  
g_pImmediateContext->OMSetRenderTargets(1, &g_pStageMasterRTV, NULL);  
g_pImmediateContext->PSSetShader(g_pPixelShaderFinal, NULL, 0);  
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
```

```
if (g_bUseCPUProcessing && g_pCpuTextureSRV) {  
    g_pImmediateContext->PSSetShaderResources(2, 1, &g_pCpuTextureSRV);  
}  
else {  
    g_pImmediateContext->PSSetShaderResources(2, 1, &g_pStage3SRV);  
}  
g_pImmediateContext->Draw(4, 0);
```

```
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);  
g_pImmediateContext->PSSetShaderResources(2, 1, nullSRV);  
g_pImmediateContext->OMSetRenderTargets(1, nullRTV, NULL);  
}
```

```

    if (g_hDlgWnd) InvalidateRect(g_hDlgWnd, NULL, FALSE);

    g_bNeedsUpdate = false; // Тяжелый расчет сбросили

    if (g_hDlgStatusWnd) SendMessage(g_hDlgStatusWnd, SB_SETTEXT, 0, (LPARAM)L"Фильтрация
завершена.");
}

// =====
// ЗОНА ДИНАМИЧЕСКОЙ ГЕОМЕТРИИ (МАСШТАБ, ПОЛЗУНКИ, ШТОРКА): ВЫЗЫВАЕТСЯ ВСЕГДА!
// =====

float xLeft = -1.0f, xRight = 1.0f;
float yTop = 1.0f, yBottom = -1.0f;

if (g_bFitToWindow)
{
    float windowAspect = (float)g_wndW / (float)g_wndH;
    float imageAspect = renderW / renderH;
    if (imageAspect > windowAspect) {
        float scaleY = windowAspect / imageAspect;
        yTop = scaleY; yBottom = -scaleY;
    }
    else {
        float scaleX = imageAspect / windowAspect;
        xLeft = -scaleX; xRight = scaleX;
    }
}
else
{
    // Сюда мгновенно транслируются изменения от ползунков g_scrollRatioX/Y и масштаба
    g_zoomDivider

    float targetW = renderW / g_zoomDivider;
    float targetH = renderH / g_zoomDivider;

    float maxScrollX = (targetW > (float)g_wndW) ? (targetW - (float)g_wndW) : 0.0f;
    float maxScrollY = (targetH > (float)g_wndH) ? (targetH - (float)g_wndH) : 0.0f;

```

```

float currentOffsetX = g_scrollRatioX * maxScrollX;
float currentOffsetY = g_scrollRatioY * maxScrollY;
float posX = -currentOffsetX;
float posY = -currentOffsetY;
xLeft = (posX / (float)g_wndW) * 2.0f - 1.0f;
xRight = ((posX + targetW) / (float)g_wndW) * 2.0f - 1.0f;
yTop = 1.0f - (posY / (float)g_wndH) * 2.0f;
yBottom = 1.0f - ((posY + targetH) / (float)g_wndH) * 2.0f;
}

// Каждую микросекунду обновляем координаты сетки на видеокарте
g_currentVertices[0] = { { xLeft, yTop, 0.0f }, { 0.0f, 0.0f } };
g_currentVertices[1] = { { xRight, yTop, 0.0f }, { 1.0f, 0.0f } };
g_currentVertices[2] = { { xLeft, yBottom, 0.0f }, { 0.0f, 1.0f } };
g_currentVertices[3] = { { xRight, yBottom, 0.0f }, { 1.0f, 1.0f } };
g_pImmediateContext->UpdateSubresource(g_pVertexBuffer, 0, NULL, g_currentVertices, 0, 0);

// =====
// ЗОНА МГНОВЕННОГО ВЫВОДА КАДРА НА ЭКРАН (60+ FPS)
// =====
float ClearColorGrey[] = { 0.75f, 0.75f, 0.75f, 1.0f };
g_pImmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearColorGrey);
g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

D3D11_VIEWPORT vpFull = { 0.0f, 0.0f, (float)g_wndW, (float)g_wndH, 0.0f, 1.0f };
g_pImmediateContext->RSSetViewports(1, &vpFull);
g_pImmediateContext->PSSetConstantBuffers(0, 1, &g_pConstantBuffer);
g_pImmediateContext->PSSetSamplers(0, 1, &g_pSamplerState);

g_pImmediateContext->PSSetShader(g_pPixelShaderCopy, NULL, 0);
if (g_pStageMasterSRV) {
    g_pImmediateContext->PSSetShaderResources(0, 1, &g_pStageMasterSRV);
}

```

```

    }

    else {
        g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
    }

    g_pImmediateContext->Draw(4, 0);
    g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);

    g_pSwapChain->Present(0, 0);
}

*/

void RenderSplit()
{
    if (g_pRenderTargetView == NULL) return;

    // 1. Общие настройки сетки экрана
    UINT stride = sizeof(SimpleVertex);
    UINT offset = 0;

    g_pImmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);
    g_pImmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);
    g_pImmediateContext->IASetInputLayout(g_pVertexLayout);

    // 2. Получаем размеры оригинального файла для шейдеров
    ShaderConstants cbData;
    cbData.width = 2.0f;
    cbData.height = 2.0f;
    if (g_pTextureSRV)
    {
        ID3D11Resource* pRes = NULL;
        g_pTextureSRV->GetResource(&pRes);
        if (pRes)

```

```

{
    ID3D11Texture2D* pT2D = (ID3D11Texture2D*)pRes;
    D3D11_TEXTURE2D_DESC d;
    pT2D->GetDesc(&d);
    cbData.width = (float)d.Width;
    cbData.height = (float)d.Height;
    pRes->Release();
}
}

cbData.d_width = 1.0f / cbData.width;
cbData.d_height = 1.0f / cbData.height;
g_pImmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbData, 0, 0);


g_pImmediateContext->VSSetShader(g_pVertexShader, NULL, 0);
g_pImmediateContext->PSSetConstantBuffers(0, 1, &g_pConstantBuffer);
g_pImmediateContext->PSSetSamplers(0, 1, &g_pSamplerState);


ID3D11ShaderResourceView* nullSRV[] = { NULL };


// --- ВЫПОЛНЯЕМ СКРЫТЫЕ ПРОХОДЫ АНАЛИЗА (1, 2 и 3) ---
float ClearBlack[] = { 0.0f, 0.0f, 0.0f, 1.0f };


// Проход 1 (Stage1)
g_pImmediateContext->ClearRenderTargetView(g_pStage1RTV, ClearBlack);
g_pImmediateContext->OMSetRenderTargets(1, &g_pStage1RTV, NULL);
g_pImmediateContext->PSSetShader(g_pPixelShader, NULL, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
g_pImmediateContext->Draw(4, 0);


// Проход 2 (Stage2)
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_pImmediateContext->ClearRenderTargetView(g_pStage2RTV, ClearBlack);

```

```
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage2RTV, NULL);  
g_plmmediateContext->PSSetShader(g_pPixelShaderStage2, NULL, 0);  
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pStage1SRV);  
g_plmmediateContext->Draw(4, 0);
```

```
// Проход 3 (Stage3)
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);  
g_plmmediateContext->ClearRenderTargetView(g_pStage3RTV, ClearBlack);  
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage3RTV, NULL);  
g_plmmediateContext->PSSetShader(g_pPixelShaderStage3, NULL, 0);  
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pStage2SRV);  
g_plmmediateContext->Draw(4, 0);
```

```
// --- ВЫВОД НА ЭКРАН В ДВА ПОРТА ПРОСМОТРА ---
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);  
float ClearGrey[] = { 0.75f, 0.75f, 0.75f, 1.0f };  
g_plmmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearGrey);  
g_plmmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);
```

```
// Жесткие безопасные размеры для деления окна пополам
```

```
float sideWidth = 400.0f;  
float sideHeight = 600.0f;
```

```
// ЛЕВАЯ ПОЛОВИНА: Сырой оригинал
```

```
D3D11_VIEWPORT vpL = {};  
vpL.Width = sideWidth;  
vpL.Height = sideHeight;  
vpL.MaxDepth = 1.0f;  
g_plmmediateContext->RSSetViewports(1, &vpL);
```

```
g_plmmediateContext->PSSetShader(g_pPixelShader, NULL, 0);  
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
```

```

g_plmmediateContext->Draw(4, 0);

g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);


// ПРАВАЯ ПОЛОВИНА: Финальный сглаженный результат из Stage3
D3D11_VIEWPORT vpR = {};
vpR.Width = sideWidth;
vpR.Height = sideHeight;
vpR.TopLeftX = sideWidth; // Сдвиг вправо
vpR.MaxDepth = 1.0f;
g_plmmediateContext->RSSetViewports(1, &vpR);


g_plmmediateContext->PSSetShader(g_pPixelShaderFinal, NULL, 0);
// >>> ВНЕДРЯЕМ АДАПТИВНЫЙ ВЫБОР: GPU ИЛИ КРАСИВАЯ КАРТИНКА CPU
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV); // Слева Оригинал


// Гибкое переключение источника для правой половины шторки
if (g_bUseCPUProcessing && g_pCpuTextureSRV)
{
    // Включен Программный режим — подаем текстуру, рассчитанную процессором
    g_plmmediateContext->PSSetShaderResources(1, 1, &g_pCpuTextureSRV);
}
else
{
    // Включен аппаратный режим — подаем результат Stage3 от видеокарты
    g_plmmediateContext->PSSetShaderResources(1, 1, &g_pStage3SRV);
}


// Сброс и вывод кадра
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_pSwapChain->Present(0, 0);
}

```

// Безопасное освобождение памяти при выходе

void CleanupDevice()

{

if (g_pImmediateContext) g_pImmediateContext->ClearState();

if (g_pStage1SRV) { g_pStage1SRV->Release(); g_pStage1SRV = NULL; }

if (g_pStage1RTV) { g_pStage1RTV->Release(); g_pStage1RTV = NULL; }

if (g_pStage2SRV) { g_pStage2SRV->Release(); g_pStage2SRV = NULL; } // НОВОЕ: Очистка ресурса чтения Stage2

if (g_pStage2RTV) { g_pStage2RTV->Release(); g_pStage2RTV = NULL; } // НОВОЕ: Очистка холста записи Stage2

if (g_pStage3SRV) { g_pStage3SRV->Release(); g_pStage3SRV = NULL; }

if (g_pStage3RTV) { g_pStage3RTV->Release(); g_pStage3RTV = NULL; }

if (g_pStageMasterSRV) { g_pStageMasterSRV->Release(); g_pStageMasterSRV = NULL; }

if (g_pStageMasterRTV) { g_pStageMasterRTV->Release(); g_pStageMasterRTV = NULL; }

// Чистое освобождение ресурсов программного режима CPU

if (g_pCpuTextureSRV) { g_pCpuTextureSRV->Release(); g_pCpuTextureSRV = NULL; }

if (g_pCpuTexture) { g_pCpuTexture->Release(); g_pCpuTexture = NULL; }

if (g_pSamplerState) { g_pSamplerState->Release(); g_pSamplerState = NULL; }

if (g_pConstantBuffer) { g_pConstantBuffer->Release(); g_pConstantBuffer = NULL; }

if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }

if (g_pVertexBuffer) { g_pVertexBuffer->Release(); g_pVertexBuffer = NULL; }

if (g_pVertexLayout) { g_pVertexLayout->Release(); g_pVertexLayout = NULL; }

if (g_pPixelShader) { g_pPixelShader->Release(); g_pPixelShader = NULL; }

if (g_pPixelShaderStage2) { g_pPixelShaderStage2->Release(); g_pPixelShaderStage2 = NULL; } // НОВОЕ: Очистка 4-го шейдера анализа

if (g_pPixelShaderStage3) { g_pPixelShaderStage3->Release(); g_pPixelShaderStage3 = NULL; } // НОВОЕ: Очистка 5-го шейдера фильтрации

if (g_pPixelShaderFinal) { g_pPixelShaderFinal->Release(); g_pPixelShaderFinal = NULL; } // НОВОЕ: Очистка 6-го шейдера цветокоррекции

if (g_pPixelShaderCopy) { g_pPixelShaderCopy->Release(); g_pPixelShaderCopy = NULL; }

if (g_pVertexShader) { g_pVertexShader->Release(); g_pVertexShader = NULL; }

if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }

if (g_pSwapChain) { g_pSwapChain->Release(); g_pSwapChain = NULL; }

```

if (g_plmmediateContext) { g_plmmediateContext->Release(); g_plmmediateContext = NULL; }
if (g_pd3dDevice) { g_pd3dDevice->Release(); g_pd3dDevice = NULL; }
if (pCompressedBuffer8x != NULL) {
    delete[] pCompressedBuffer8x;
    pCompressedBuffer8x = NULL;
}
if (pStretchedBuffer != NULL) {
    delete[] pStretchedBuffer;
    pStretchedBuffer = NULL;
}
}

```

// === НОВЫЙ КОД: ФУНКЦИЯ СОЗДАНИЯ СИСТЕМНОГО МЕНЮ ===

```
void CreateAppMenu(HWND hwnd)
```

```

{
    HMENU hMenu = CreateMenu();    // Главная горизонтальная полоса
    HMENU hFileMenu = CreatePopupMenu(); // Выпадающее меню "Файл"
    HMENU hViewMenu = CreatePopupMenu(); // Выпадающее меню "Вид"
    HMENU hHelpMenu = CreatePopupMenu(); // Выпадающее меню "Справка"

    // Заполняем меню "Файл"
    AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_VIDEO, L"Открыть видео поток (AVI/MP4)...");
    AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_IMAGE, L"Открыть кадр/картинку (BMP/PNG)...");

    // === НОВОЕ: ДОБАВЛЯЕМ УПРАВЛЕНИЕ РЕЖИМАМИ СРАВНЕНИЯ ===
    AppendMenuW(hViewMenu, MF_STRING, IDM_VIEW_CPU, L"Программный режим (CPU)");

    // === ВНЕДРЕНИЕ: КАСКАДНОЕ ПОДМЕНЮ ДИСКРЕТНЫХ МАСШТАБОВ ===
    g_hZoomSubMenu = CreateMenu();

    // Формируем 6 эталонных вариантов масштабирования
    AppendMenuW(g_hZoomSubMenu, MF_STRING, IDM_ZOOM_FIT, L"Вписать в кадр");
    AppendMenuW(g_hZoomSubMenu, MF_STRING, IDM_ZOOM_1, L"Масштаб 1:1 (Оригинал)");
    AppendMenuW(g_hZoomSubMenu, MF_STRING, IDM_ZOOM_2, L"Масштаб 1:2");
}

```

```

AppendMenuW(g_hZoomSubMenu, MF_STRING, IDM_ZOOM_4, L"Масштаб 1:4");
AppendMenuW(g_hZoomSubMenu, MF_STRING, IDM_ZOOM_8, L"Масштаб 1:8");
AppendMenuW(g_hZoomSubMenu, MF_STRING, IDM_ZOOM_16, L"Масштаб 1:16");

// Встраиваем всплывающее подменю Масштаба внутрь нашего родительского меню "Вид"
[INDEX_146]
AppendMenuW(hViewMenu, MF_POPUP, (UINT_PTR)g_hZoomSubMenu, L"Масштаб");

AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hViewMenu, L"Вид");
AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL); // Разделительная линия
AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_SAVE_RESULT, L"Записать преобразованный
файл...");
AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL);
AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_EXIT, L"Выход");

// Заполняем меню "Вид"
AppendMenuW(hViewMenu, MF_STRING | MF_CHECKED, IDM_VIEW_RESULT_ONLY, L"Только
обработанный результат");
AppendMenuW(hViewMenu, MF_STRING, IDM_VIEW_SPLIT, L"Режим \"До / После\" (Разделение
экрана)");

// Заполняем меню "Справка"
AppendMenuW(hHelpMenu, MF_STRING, IDM_HELP_ABOUT, L"О программе...");

// Прикрепляем выпадающие списки к главной полосе меню
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"Файл");
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hViewMenu, L"Вид");
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Справка");

// Физически вешаем меню на наше окно программы
SetMenu(hwnd, hMenu);
}

// === КОНЕЦ НОВОГО КОДА ===

```

```

void ProcessAndAllocateBuffers(DWORD* pInitialPixels, UINT width, UINT height)
{
    // 0. Освобождаем старую память, если она была выделена
    delete[] g_pBuffer1_Src;
    delete[] g_pBuffer2_Padded;
    delete[] g_pBuffer3_Monochrome;
    delete[] g_pBuffer4_Compressed8x;

    g_srcImgWidth = width;
    g_srcImgHeight = height;

    // 1. Заполняем БУФЕР 1: Оригинал
    UINT srcSize = g_srcImgWidth * g_srcImgHeight;
    g_pBuffer1_Src = new DWORD[srcSize];
    memcpy(g_pBuffer1_Src, pInitialPixels, srcSize * sizeof(DWORD));

    // Вычисляем размеры с округлением вверх до кратности 8
    g_paddedImgWidth = ((g_srcImgWidth + 7) / 8) * 8;
    g_paddedImgHeight = ((g_srcImgHeight + 7) / 8) * 8;
    UINT paddedSize = g_paddedImgWidth * g_paddedImgHeight;

    // 2. Создаем БУФЕР 2: Дополненный до кратности 8 (заливаем черным)
    g_pBuffer2_Padded = new DWORD[paddedSize];
    memset(g_pBuffer2_Padded, 0, paddedSize * sizeof(DWORD));

    // Копируем оригинал внутрь дополненного буфера
    for (UINT y = 0; y < g_srcImgHeight; y++)
    {
        memcpy(&g_pBuffer2_Padded[y * g_paddedImgWidth],
            &g_pBuffer1_Src[y * g_srcImgWidth],
            g_srcImgWidth * sizeof(DWORD));
    }
}

```

```
}
```

```
// 3. Создаем БУФЕР 3: Монохромный (на основе дополненного кадра)
```

```
g_pBuffer3_Monochrome = new DWORD[paddedSize];
```

```
for (UINT i = 0; i < paddedSize; i++)
```

```
{
```

```
    DWORD color = g_pBuffer2_Padded[i];
```

```
    BYTE b = (BYTE)(color & 0xFF);
```

```
    BYTE g = (BYTE)((color >> 8) & 0xFF);
```

```
    BYTE r = (BYTE)((color >> 16) & 0xFF);
```

```
    // Ваша точная формула перевода в серое вещество
```

```
    BYTE gray = (BYTE)(r * 0.299f + g * 0.587f + b * 0.114f);
```

```
    // Сохраняем как честный 32-битный серый пиксель (0x00RRGGBB)
```

```
    g_pBuffer3_Monochrome[i] = RGB(gray, gray, gray);
```

```
}
```

```
// Размеры сжатого кадра
```

```
g_compressedWidth = g_paddedImgWidth / 8;
```

```
g_compressedHeight = g_paddedImgHeight / 8;
```

```
UINT compressedSize = g_compressedWidth * g_compressedHeight;
```

```
// 4. Создаем БУФЕР 4: Сжатый в 8 раз (Режим 3)
```

```
g_pBuffer4_Compressed8x = new DWORD[compressedSize];
```

```
// Алгоритм сжатия блоков 8x8 (записывает среднее или дефектное значение блока)
```

```
for (UINT py = 0; py < g_compressedHeight; py++)
```

```
{
```

```
    for (UINT px = 0; px < g_compressedWidth; px++)
```

```
    {
```

```
        // Здесь ваша логика обработки блока 8x8 пикселей из g_pBuffer3_Monochrome.
```

```

// Для примера возьмем просто левый верхний пиксель каждого макроблока:

UINT srcX = px * 8;

UINT srcY = py * 8;

DWORD blockColor = g_pBuffer3_Monochrome[srcY * g_paddedImgWidth + srcX];


    g_pBuffer4_Compressed8x[py * g_compressedWidth + px] = blockColor;
}
}
}

#include <wincodec.h> // Подключаем заголовки системного декодера WIC

// === БЕЗОПАСНАЯ ЗАГРУЗКА ЛЮБЫХ КАРТИНОК ЧЕРЕЗ WIC НА ВИДЕОКАРТУ ===
HRESULT LoadTextureFromFile(const WCHAR* szFileName)
{
    HRESULT hr = S_OK;

    // 1. Создаем фабрику декодеров WIC
    IWICImagingFactory* pWICFactory = NULL;

    hr = CoCreateInstance(CLSID_WICImagingFactory, NULL, CLSCTX_INPROC_SERVER,
        __uuidof(IWICImagingFactory), (LPVOID*)&pWICFactory);

    if (FAILED(hr)) return hr;

    // 2. Открываем файл картинки на диске
    IWICBitmapDecoder* pDecoder = NULL;

    hr = pWICFactory->CreateDecoderFromFilename(szFileName, NULL, GENERIC_READ,
        WICDecodeMetadataCacheOnDemand, &pDecoder);

    if (FAILED(hr)) { pWICFactory->Release(); return hr; }

    // 3. Берем самый первый кадр из файла
    IWICBitmapFrameDecode* pFrame = NULL;

    hr = pDecoder->GetFrame(0, &pFrame);

    if (FAILED(hr)) { pDecoder->Release(); pWICFactory->Release(); return hr; }
}

```

```

// 4. Принудительно конвертируем пиксели в формат RGBA 32-бит
IWICFormatConverter* pConverter = NULL;

hr = pWICFactory->CreateFormatConverter(&pConverter);

if (FAILED(hr)) { pFrame->Release(); pDecoder->Release(); pWICFactory->Release(); return hr; }

hr = pConverter->Initialize(pFrame, GUID_WICPixelFormat32bppPBGRA, WICBitmapDitherTypeNone,
NULL, 0.0f, WICBitmapPaletteTypeCustom);

if (FAILED(hr)) { pConverter->Release(); pFrame->Release(); pDecoder->Release(); pWICFactory-
>Release(); return hr; }

// 5. Узнаем физические размеры загруженной картинки
UINT imgWidth = 0, imgHeight = 0;

pConverter->GetSize(&imgWidth, &imgHeight);

if (pPixelsBuffer != NULL)
{
    delete[] pPixelsBuffer; // Физически возвращаем операционной системе старый блок
    pPixelsBuffer = NULL; // Сразу зануляем ярлык, чтобы никто не полез в пустую память
}

if (pRawPixels != NULL)
{
    delete[] pRawPixels; // Физически возвращаем операционной системе старый блок
    pRawPixels = NULL; // Сразу зануляем ярлык, чтобы никто не полез в пустую память
}

// Вычисляем новые размеры, кратные 8
UINT paddedWidth = ((imgWidth + 7) / 8) * 8;
UINT paddedHeight = ((imgHeight + 7) / 8) * 8;

pPixelsBuffer = new UINT[paddedWidth * paddedHeight];

// Вычисляем честный stride: для формата 32bpp это ровно imgWidth * 4 байт
UINT wicStride = imgWidth * sizeof(UINT);

```

```

pRawPixels = new UINT[imgWidth * imgHeight];

// Копируем пиксели из WIC, используя правильный stride строки
hr = pConverter->CopyPixels(NULL, wicStride, wicStride * imgHeight, (BYTE*)pRawPixels);
if (SUCCEEDED(hr))
{
    // Заполняем расширенный буфер с дублированием краев
    for (UINT y = 0; y < paddedHeight; y++)
    {
        // Если вышли за нижний край оригинала — дублируем последнюю строку кадра
        UINT srcY = (y < imgHeight) ? y : (imgHeight - 1);

        for (UINT x = 0; x < paddedWidth; x++)
        {
            // Если вышли за правый край оригинала — дублируем последний пиксель строки
            UINT srcX = (x < imgWidth) ? x : (imgWidth - 1);

            // ВАЖНО: Читаем из сырого буфера по шагу wicStride (imgWidth),
            // а пишем в расширенный по шагу paddedWidth!
            pPixelsBuffer[y * paddedWidth + x] = pRawPixels[srcY * imgWidth + srcX];
        }
    }

    // 6. Если старая основная текстура уже была в памяти — чисто освобождаем её
    if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }

    // 7. Создаем новую текстуру прямо в видеопамяти под размеры нашей картинки
    D3D11_TEXTURE2D_DESC desc = {};
    desc.Width = imgWidth;
    desc.Height = imgHeight;
    desc.MipLevels = 1;
    desc.ArraySize = 1;

```

```

desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;

desc.SampleDesc.Count = 1;

desc.Usage = D3D11_USAGE_DEFAULT;

desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;


D3D11_SUBRESOURCE_DATA initData = {};

initData.pSysMem = pPixelsBuffer;

initData.SysMemPitch = paddedWidth * sizeof(UINT); // Задаем видеокарте шаг 184


ID3D11Texture2D* pTexture = NULL;

desc.Width = paddedWidth; // Задаем ширину текстуры 184

desc.Height = paddedHeight; // Задаем высоту текстуры 184

hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);

if (SUCCEEDED(hr) && g_pTextureStaging && pTexture)
{
    // Видеокарта делает моментальный слепок оригинального кадра в наш шпион со всеми
    // нужными флагами!

    g_pImmediateContext->CopyResource(g_pTextureStaging, pTexture);
}

if (SUCCEEDED(hr))
{
    hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);

    pTexture->Release();


    g_currentImgWidth = paddedWidth; // Строго paddedWidth (184)! Никаких imgWidth.

    g_currentImgHeight = paddedHeight; // Строго paddedHeight (184)!


    g_bFitToWindow = true; // Подтверждаем режим авто-вписывания в памяти

    PostMessage(hwnd, WM_COMMAND, IDM_ZOOM_FIT, 0); // Кидаем команду "Вписать в кадр"
    // в очередь
}

// НАЧАЛО ВСТАВКИ: Гарантированный расчет сжатия, когда все глобальные переменные уже
// заполнены

```

```

if (pPixelsBuffer != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
{
    if (pCompressedBuffer8x != NULL)
    {
        delete[] pCompressedBuffer8x;
        pCompressedBuffer8x = NULL;
    }

    UINT compressedW = g_currentImgWidth / 8;
    UINT compressedH = g_currentImgHeight / 8;

    if (compressedW > 0 && compressedH > 0)
    {
        pCompressedBuffer8x = new UINT[compressedW * compressedH];

        for (UINT py = 0; py < compressedH; py++)
        {
            for (UINT px = 0; px < compressedW; px++)
            {
                UINT sumR = 0, sumG = 0, sumB = 0;

                // Пробегаем по внутреннему блоку 8x8 исходных пикселей
                for (UINT by = 0; by < 8; by++)
                {
                    for (UINT bx = 0; bx < 8; bx++)
                    {
                        UINT srcX = px * 8 + bx;
                        UINT srcY = py * 8 + by;

                        // Читаем из уже готового и проверенного глобального массива
                        DWORD color = pPixelsBuffer[srcY * g_currentImgWidth + srcX];

```

```

        sumB += (color & 0xFF);
        sumG += ((color >> 8) & 0xFF);
        sumR += ((color >> 16) & 0xFF);
    }
}

BYTE avgB = (BYTE)(sumB / 64);
BYTE avgG = (BYTE)(sumG / 64);
BYTE avgR = (BYTE)(sumR / 64);

// Монохром по формуле дефектоскопа
float grayVal = (float)avgR * 0.299f + (float)avgG * 0.587f + (float)avgB * 0.114f;
BYTE gray = (BYTE)grayVal;

pCompressedBuffer8x[py * compressedW + px] = gray | (gray << 8) | (gray << 16) | (0xFF
<< 24);
    }
}
}
}

// КОНЕЦ ВСТАВКИ
if (SUCCEEDED(hr) && g_hDlgWnd != NULL)
{
    UpdateDlgScrollbars(g_hDlgWnd); // Сразу настраиваем ползунки под текущий режим
}

// === НОВЫЙ КОД: ЗАПУСК ПРОГРАММНОЙ ОБРАБОТКИ CPU ПРИ ОТКРЫТИИ ФАЙЛА ===
if (SUCCEEDED(hr))
{
    UINT* pCpuOutPixels = new UINT[paddedWidth * paddedHeight];

    // Запускаем расчет на процессоре по НАСТОЯЩЕМУ исходному размеру кадра
    if (SUCCEEDED(ApplyCpuFilter(pPixelsBuffer, paddedWidth, paddedHeight, pCpuOutPixels)))

```

```

{
    // Описываем параметры для текстуры процессора (тоже строго 184x184)
    D3D11_TEXTURE2D_DESC cpuDesc = {};
    cpuDesc.Width = paddedWidth;
    cpuDesc.Height = paddedHeight;
    cpuDesc.MipLevels = 1;
    cpuDesc.ArraySize = 1;
    cpuDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    cpuDesc.SampleDesc.Count = 1;
    cpuDesc.Usage = D3D11_USAGE_DEFAULT;
    cpuDesc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

    D3D11_SUBRESOURCE_DATA cpuInitData = {};
    cpuInitData.pSysMem = pCpuOutPixels;
    cpuInitData.SysMemPitch = paddedWidth * sizeof(UINT);

    // Физически создаем холст результатов CPU на видеокарте
    ID3D11Texture2D* pCpuTex = NULL;
    if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&cpuDesc, &cpuInitData, &pCpuTex)))
    {
        g_pd3dDevice->CreateShaderResourceView(pCpuTex, NULL, &g_pCpuTextureSRV);
        pCpuTex->Release();
    }
}

// Освобождаем временный массив результатов процессора из ОЗУ
delete[] pCpuOutPixels;
}

// =====
// СИНХРОНИЗАЦИЯ С КОДОМ 2011 ГОДА: ДИНАМИЧЕСКИЙ РЕЗАЙЗ СКРЫТЫХ БУФЕРОВ
// =====

```

```

if (SUCCEEDED(hr))
{
    // Чисто освобождаем все старые скрытые буферы из видеопамати [INDEX_45, INDEX_46]
    if (g_pStage1RTV) { g_pStage1RTV->Release(); g_pStage1RTV = NULL; }
    if (g_pStage1SRV) { g_pStage1SRV->Release(); g_pStage1SRV = NULL; }
    if (g_pStage2RTV) { g_pStage2RTV->Release(); g_pStage2RTV = NULL; }
    if (g_pStage2SRV) { g_pStage2SRV->Release(); g_pStage2SRV = NULL; }
    if (g_pStage3RTV) { g_pStage3RTV->Release(); g_pStage3RTV = NULL; }
    if (g_pStage3SRV) { g_pStage3SRV->Release(); g_pStage3SRV = NULL; }

    // Вычисляем правильные пирамидальные размеры сжатия [INDEX_79]
    UINT stage1Width = (imgWidth / 4 > 0) ? imgWidth / 4 : 1;
    UINT stage1Height = (imgHeight / 4 > 0) ? imgHeight / 4 : 1;

    UINT stage2Width = (imgWidth / 8 > 0) ? imgWidth / 8 : 1;
    UINT stage2Height = (imgHeight / 8 > 0) ? imgHeight / 8 : 1;

    // --- Создаем буфер Stage 1 (Меньше в 4 раза под блоки 4x4) ---
    D3D11_TEXTURE2D_DESC descS1 = {};
    descS1.Width = stage1Width;
    descS1.Height = stage1Height;
    descS1.MipLevels = 1;
    descS1.ArraySize = 1;
    descS1.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    descS1.SampleDesc.Count = 1;
    descS1.Usage = D3D11_USAGE_DEFAULT;
    descS1.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

    ID3D11Texture2D* pTexS1 = NULL;
    if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS1, NULL, &pTexS1))) {
        g_pd3dDevice->CreateRenderTargetView(pTexS1, NULL, &g_pStage1RTV);
        g_pd3dDevice->CreateShaderResourceView(pTexS1, NULL, &g_pStage1SRV);
    }
}

```

```
pTexS1->Release();  
}
```

```
// --- Создаем буфер Stage 2 (Меньше в 8 раз под макроблоки 8x8) ---
```

```
D3D11_TEXTURE2D_DESC descS2 = {};  
descS2.Width = stage2Width;  
descS2.Height = stage2Height;  
descS2.MipLevels = 1;  
descS2.ArraySize = 1;  
descS2.Format = DXGI_FORMAT_R8G8B8A8_UNORM;  
descS2.SampleDesc.Count = 1;  
descS2.Usage = D3D11_USAGE_DEFAULT;  
descS2.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
```

```
ID3D11Texture2D* pTexS2 = NULL;
```

```
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS2, NULL, &pTexS2))) {  
    g_pd3dDevice->CreateRenderTargetView(pTexS2, NULL, &g_pStage2RTV);  
    g_pd3dDevice->CreateShaderResourceView(pTexS2, NULL, &g_pStage2SRV);  
    pTexS2->Release();  
}
```

```
// --- Создаем буфер Stage 3 (ПОЛНОРАЗМЕРНЫЙ под гладкий финальный кадр) ---
```

```
// --- ИСПРАВЛЕНИЕ: ПЕРЕВОДИМ STAGE 3 В НАСТОЯЩИЙ HDR-ФОРМАТ С ПЛАВАЮЩЕЙ  
ТОЧКОЙ ---
```

```
D3D11_TEXTURE2D_DESC descS3 = {};  
descS3.Width = imgWidth;  
descS3.Height = imgHeight;  
descS3.MipLevels = 1;  
descS3.ArraySize = 1;  
descS3.Format = DXGI_FORMAT_R16G16B16A16_FLOAT; // ВКЛЮЧИЛИ 16-БИТНЫЙ FLOAT ДЛЯ  
МАТЕМАТИКИ ИЗ 2011 ГОДА!  
descS3.SampleDesc.Count = 1;  
descS3.Usage = D3D11_USAGE_DEFAULT;
```

```

descS3.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pTexS3 = NULL;
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS3, NULL, &pTexS3))) {
    g_pd3dDevice->CreateRenderTargetView(pTexS3, NULL, &g_pStage3RTV);
    g_pd3dDevice->CreateShaderResourceView(pTexS3, NULL, &g_pStage3SRV);
    pTexS3->Release();
}
}

if (g_pStageMasterRTV) { g_pStageMasterRTV->Release(); g_pStageMasterRTV = NULL; }
if (g_pStageMasterSRV) { g_pStageMasterSRV->Release(); g_pStageMasterSRV = NULL; }

D3D11_TEXTURE2D_DESC descMaster = {};
descMaster.Width = imgWidth;
descMaster.Height = imgHeight;
descMaster.MipLevels = 1;
descMaster.ArraySize = 1;
descMaster.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
descMaster.SampleDesc.Count = 1;
descMaster.Usage = D3D11_USAGE_DEFAULT;
descMaster.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pTexMaster = NULL;
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descMaster, NULL, &pTexMaster))) {
    g_pd3dDevice->CreateRenderTargetView(pTexMaster, NULL, &g_pStageMasterRTV);
    g_pd3dDevice->CreateShaderResourceView(pTexMaster, NULL, &g_pStageMasterSRV);
    pTexMaster->Release();
}
}

// === ИСПРАВЛЕНИЕ: ЖЕСТКОЕ И БЕЗОПАСНОЕ ИЗМЕНЕНИЕ РАЗМЕРОВ ОКНА ПРИ ОТКРЫТИИ
ФАЙЛА ===
if (SUCCEEDED(hr))
{

```

```

HWND hMainWnd = FindWindowW(L"MyShaderVideoPlayerClass", NULL);

if (hMainWnd)
{
    // Рассчитываем габариты окна с учетом меню и заголовка Windows
    RECT rc = { 0, 0, (LONG)imgWidth, (LONG)imgHeight };
    AdjustWindowRect(&rc, WS_OVERLAPPEDWINDOW, TRUE);

    // Физически раздвигаем окно под размер картинки!
    MoveWindow(hMainWnd, 100, 100, rc.right - rc.left, rc.bottom - rc.top, TRUE);
}
}

// =====
// НАСТРОЙКА ДИАПАЗОНОВ С КРОЛЛИНГА ПОД РАЗМЕР КАРТИНКИ
// =====

if (SUCCEEDED(hr))
{
    // Сохраняем размеры файла для обработчика прокрутки
    g_currentImgWidth = imgWidth;
    g_currentImgHeight = imgHeight;
    g_scrollX = 0;
    g_scrollY = 0;

    // === АВТОМАТИЧЕСКИЙ РАСЧЕТ КОЭФФИЦИЕНТОВ СКОРОСТИ СКРОЛЛИНГА ===
    g_scrollMultiplierX = (imgWidth > (UINT)g_wndW) ? ((float)imgWidth / g_wndW) : 1.0f;
    g_scrollMultiplierY = (imgHeight > (UINT)g_wndH) ? ((float)imgHeight / g_wndH) : 1.0f;

    HWND hMainWnd = FindWindowW(L"MyShaderVideoPlayerClass", NULL);
    if (hMainWnd)
    {
        // Настраиваем горизонтальный ползунок (X)
        SCROLLINFO siX = {};

```

```

    siX.cbSize = sizeof(SCROLLINFO);

    siX.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;

    siX.nMin = 0;

    siX.nMax = imgWidth;

    siX.nPage = 800; // Шаг страницы равен размеру окна

    siX.nPos = 0;

    SetScrollInfo(hMainWnd, SB_HORZ, &siX, TRUE);

    // Настраиваем вертикальный ползунок (Y)
    SCROLLINFO siY = {};

    siY.cbSize = sizeof(SCROLLINFO);

    siY.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;

    siY.nMin = 0;

    siY.nMax = imgHeight;

    siY.nPage = 600;

    siY.nPos = 0;

    SetScrollInfo(hMainWnd, SB_VERT, &siY, TRUE);
}

}

// === ЗОЛОТОЕ ПРАВИЛО: ЧИСТО ЗАКРЫВАЕМ ВСЕ РЕСУРСЫ И ПАМЯТЬ БЕЗ УТЕЧЕК! ===
pConverter->Release();
pFrame->Release();
pDecoder->Release();
pWICFactory->Release();

return hr;
}

// === НОВЫЙ КОД: ФУНКЦИЯ ВЫЗОВА СИСТЕМНОГО ПРОВОДНИКА WINDOWS ===
bool OpenFileDialog(HWND hwnd, bool bOpenVideo)
{

```

```

OPENFILENAMEW ofn = {};

// =====

// >>> ИСПРАВЛЯЕМ СМЕРТЕЛЬНУЮ ЛОВУШКУ ДЛЯ NTDLL.DLL: ДЕЛАЕМ БУФЕР СТАТИЧЕСКИМ
// =====

static wchar_t szFile[MAX_PATH];

ZeroMemory(szFile, sizeof(szFile)); // Чисто обнуляем буфер перед каждым вызовом
// =====

ofn.lStructSize = sizeof(ofn);

ofn.hwndOwner = hwnd;

ofn.lpstrFile = szFile;

ofn.nMaxFile = MAX_PATH; // Жестко задаем максимальный размер


if (bOpenVideo)
{
    // Фильтр файлов для видеопотока

    ofn.lpstrFilter = L"Видео файлы (AVI, MP4)\0*.avi;*.mp4\0Все файлы (*.*)\0*.*\0";

    ofn.lpstrTitle = L"Выберите видео поток для дефектоскопа";
}
else
{
    // Фильтр файлов для отдельных изображений

    ofn.lpstrFilter = L"Изображения (BMP, PNG, JPG)\0*.bmp;*.png;*.jpg;*.jpeg\0Все файлы (*.*)\0*.*\0";

    ofn.lpstrTitle = L"Выберите кадр/картинку для анализа";
}


ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;


// Распахиваем системное окно Проводника
if (GetOpenFileNameW(&ofn))
{
    // Если пользователь выбрал файл и нажал "Открыть" — копируем путь в нашу глобальную
    переменную

```

```

        wcsncpy_s(g_szSelectedFilePath, MAX_PATH, szFile);

        return true;
    }

    return false; // Если пользователь нажал "Отмена"
}

// === КОНЕЦ НОВОГО КОДА ===

void UpdateZoomMenuCheck(int checkedId)
{
    if (!g_hZoomSubMenu) return;

    // Перебираем все наши 6 масштабов и снимаем с них птички
    int zoomIDs[] = { IDM_ZOOM_FIT, IDM_ZOOM_1, IDM_ZOOM_2, IDM_ZOOM_4, IDM_ZOOM_8,
IDM_ZOOM_16 };
    for (int i = 0; i < 6; i++) {
        CheckMenuItem(g_hZoomSubMenu, zoomIDs[i], MF_BYCOMMAND | MF_UNCHECKED);
    }

    // Ставим птичку строго на тот масштаб, который активен
    CheckMenuItem(g_hZoomSubMenu, checkedId, MF_BYCOMMAND | MF_CHECKED);
}

// Главная точка входа Windows-приложения
int WINAPI wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ PWSTR
pCmdLine, _In_ int nCmdShow)
{
    // === НОВЫЙ КОД: ИНИЦИАЛИЗАЦИЯ СИСТЕМНОЙ БИБЛИОТЕКИ COM ДЛЯ ДВИЖКА WIC ===
    (void)CoInitializeEx(NULL, COINIT_APARTMENTTHREADED); // Запускает поддержку WIC в Windows

    const wchar_t CLASS_NAME[] = L"MyVideoPlayerWindowClass";

    WNDCLASS wc = { };

    wc.lpfnWndProc = WindowProc;

```

```

wc.hInstance = hInstance;

wc.lpszClassName = CLASS_NAME;

wc.hCursor = LoadCursor(NULL, IDC_ARROW);

RegisterClass(&wc);


hwnd = CreateWindowW(
    CLASS_NAME,          // Строка 1591: Ваша живая переменная класса со строки 1582!
    L"Мой Шейдерный Видеоплеер", // Строка 1592: Заголовок плеера в кавычках напрямую
    WS_OVERLAPPEDWINDOW | WS_CLIPCHILDREN, // Ваши родные стили встроенных ползунков
    CW_USEDEFAULT,
    0,
    800,
    600,
    NULL,
    NULL,
    hInstance,
    NULL
);

if (hwnd == NULL) return 0;


// Инициализируем общие элементы управления Windows
InitCommonControls();


// Физически создаем строку статуса, привязанную к главному окну hwnd
g_hStatusWnd = CreateStatusWindow(
    WS_CHILD | WS_VISIBLE,
    L"HDR Дефектоскоп готов к работе",
    hwnd,          // Дескриптор вашего окна плеера
    2005
);

```

```

if (g_hStatusWnd != NULL && hwnd != NULL)
{
    // 1. Аппаратно измеряем физическую высоту созданной строки статуса (как в 2004 году!)
    RECT rS = {};
    GetWindowRect(g_hStatusWnd, &rS);
    int statusHeight = rS.bottom - rS.top; // Получили чистую высоту полосы в пикселях

    // 2. Узнаем текущие полные габариты рамы главного окна плеера на экране
    RECT wndRect = {};
    GetWindowRect(hwnd, &wndRect);
    int currentW = wndRect.right - wndRect.left;
    int currentH = wndRect.bottom - wndRect.top;

    // 3. Расширяем внешнюю коробку окна строго на высоту статус-бара!
    // Плеер остаётся в своих прежних размерах, а строка статуса ложится ниже ползунков!
    SetWindowPos(hwnd, NULL, 0, 0, currentW, currentH + statusHeight, SWP_NOMOVE |
    SWP_NOZORDER | SWP_FRAMECHANGED);
}

ShowWindow(hwnd, nCmdShow);

CreateAppMenu(hwnd); // НОВОЕ: Физически включаем меню на экране сразу после показа окна

// =====
// СИНХРОНИЗАЦИЯ СТАРТОВЫХ РЕЖИМОВ И ПАМЯТИ
// =====
g_bUseCPUProcessing = true; // Включаем CPU-режим в памяти (Страница 4)
g_bFitToWindow = true;     // Включаем авто-вписывание в памяти (Страница 5)
g_bNeedsUpdate = true;     // Разрешаем тяжелый расчет в Render() (Страница 3)

// Выставляем галочки в главном горизонтальном меню
HMENU hMainMenu = GetMenu(hwnd);
if (hMainMenu)

```

```

{
    // Ставим галочку на Программный режим (IDM_VIEW_CPU = 40003, Страница 1)
    CheckMenuItem(hMainMenu, IDM_VIEW_CPU, MF_BYCOMMAND | MF_CHECKED);

    // Снимаем галочку с режима "Только результат" (IDM_VIEW_RESULT_ONLY = 1006, Страница
1)
    CheckMenuItem(hMainMenu, IDM_VIEW_RESULT_ONLY, MF_BYCOMMAND | MF_UNCHECKED);

    // Синхронизируем подменю масштабов (g_hZoomSubMenu, Страница 3)
    if (g_hZoomSubMenu)
    {
        CheckMenuItem(g_hZoomSubMenu, IDM_ZOOM_FIT, MF_BYCOMMAND | MF_CHECKED);
    }
}

CreateDiagnosticWindow(hInstance, hwnd);

if (FAILED(InitDevice(hwnd)))
{
    CleanupDevice();

    CoUninitialize(); // НОВОЕ: Чисто закрываем и освобождаем ресурсы COM-потока перед
выходом!

    return 0;
}

MSG msg = { };
while (WM_QUIT != msg.message)
{
    // === О Т Л А Д О Ч Н Ы Й   М А Я Ч О К ===

    static bool bLoopStart = true;

    if (bLoopStart) {

        //MessageBoxW(hwnd, L"Маячок Ц-1: Мы успешно вошли в бесконечный цикл обработки
сообщений wWinMain!", L"Отладка", MB_OK);
    }
}

```

```

        bLoopStart = false;
    }

    if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    else
    {
        Render();

        //RenderSplit(); // Переключили плеер в режим сравнения "До / После"
    }
}

CleanupDevice();

CoUninitialize(); // НОВОЕ: Чисто закрываем и освобождаем ресурсы COM-потока перед
выходом!

return 0;
}

// === ИСТИННАЯ ПРОГРАММНАЯ ОБРАБОТКА ИЗ 2011 ГОДА НА ЦЕНТРАЛЬНОМ ПРОЦЕССОРЕ
(CPU) ===

// === ИСТИННАЯ ПИРАМИДАЛЬНАЯ ФИЛЬТРАЦИЯ 2011 ГОДА НА ПРОЦЕССОРЕ (CPU) ===

HRESULT ApplyCpuFilter(unsigned int* pSrcPixels, unsigned int width, unsigned int height, unsigned int*
pOutPixels)
{
    if (!pSrcPixels || !pOutPixels || width == 0 || height == 0) return E_INVALIDARG;

    // Весовые коэффициенты яркости монохрома из 2011 года
    const float W_R = 0.299f;
    const float W_G = 0.587f;
    const float W_B = 0.114f;

```

```

// -----
// МАССИВ p1: Переводим исходный кадр BGRA в монохромную матрицу яркости
// -----
float* p1 = new float[width * height];
for (unsigned int i = 0; i < width * height; i++)
{
    unsigned int pixel = pSrcPixels[i];
    BYTE b = (pixel & 0x000000FF);
    BYTE g = ((pixel & 0x0000FF00) >> 8);
    BYTE r = ((pixel & 0x00FF0000) >> 16);

    p1[i] = (float)r * W_R + (float)g * W_G + (float)b * W_B;
}

// -----
// МАССИВ p2: Первый уровень пирамиды (сжатие блоков 4x4 пикселя)
// -----
unsigned int w2 = width / 4;
unsigned int h2 = height / 4;
if (w2 == 0) w2 = 1;
if (h2 == 0) h2 = 1;

float* p2 = new float[w2 * h2];

// Цикл свертки блоков 4x4
for (unsigned int y = 0; y < h2; y++)
{
    for (unsigned int x = 0; x < w2; x++)
    {
        float sum = 0.0f;

        // Сканируем квадрат 4x4 пикселя из исходного массива p1

```

```

for (unsigned int block_y = 0; block_y < 4; block_y++)
{
    unsigned int src_y = y * 4 + block_y;
    if (src_y >= height) src_y = height - 1; // Защита от выхода за край

    for (unsigned int block_x = 0; block_x < 4; block_x++)
    {
        unsigned int src_x = x * 4 + block_x;
        if (src_x >= width) src_x = width - 1;

        sum += p1[src_y * width + src_x];
    }
}

p2[y * w2 + x] = sum / 16.0f; // Среднее арифметическое блока 4x4
}
}

// -----
// МАССИВ p3: Второй уровень пирамиды (укрупнение блоков в 8x8, или 2x2 от p2)
// -----

unsigned int w3 = w2 / 2;
unsigned int h3 = h2 / 2;
if (w3 == 0) w3 = 1;
if (h3 == 0) h3 = 1;

float* p3 = new float[w3 * h3];

// Цикл свертки блоков 2x2 от предыдущего уровня p2
for (unsigned int y = 0; y < h3; y++)
{
    for (unsigned int x = 0; x < w3; x++)
    {

```

```

float sum = 0.0f;
for (unsigned int block_y = 0; block_y < 2; block_y++)
{
    unsigned int src_y = y * 2 + block_y;
    if (src_y >= h2) src_y = h2 - 1;

    for (unsigned int block_x = 0; block_x < 2; block_x++)
    {
        unsigned int src_x = x * 2 + block_x;
        if (src_x >= w2) src_x = w2 - 1;

        sum += p2[src_y * w2 + src_x];
    }
}
p3[y * w3 + x] = sum / 4.0f; // Среднее арифметическое макроблока
}

// -----
// СБОРКА И ФИНАЛЬНЫЙ ВЫВОД (Пока выводим промежуточный массив p1)
// -----

// Цикл высокоточного межполукадрового анализа и интерполяции контуров
for (unsigned int y = 0; y < height; y++)
{
    for (unsigned int x = 0; x < width; x++)
    {
        unsigned int idx = y * width + x;
        float src_gray = p1[idx]; // Исходная точка кадра

        // 1. Вычисляем внутриблочный шаг текущего пикселя (от 0 до 3)
        unsigned int i_x = x % 4;
        unsigned int i_y = y % 4;
    }
}

```

```

// Координаты центрального блока 4x4
unsigned int x2_0 = x / 4;
unsigned int y2_0 = y / 4;

// Координаты правого и нижнего соседей с защитой от вылета за границы сетки w2/h2
unsigned int x2_1 = (x2_0 + 1 < w2) ? x2_0 + 1 : x2_0;
unsigned int y2_1 = (y2_0 + 1 < h2) ? y2_0 + 1 : y2_0;

// Извлекаем яркости четырех смежных макроблоков из пирамиды p2
float k_S = p2[y2_0 * w2 + x2_0]; // Центральный
float k_LR = p2[y2_0 * w2 + x2_1]; // Справа (сбоку)
float k_TB = p2[y2_1 * w2 + x2_0]; // Снизу (вертикаль)
float k_LRTB = p2[y2_1 * w2 + x2_1]; // Справа снизу (угол)

// Масштабируем шаг пикселя под весовые коэффициенты (0, 2, 4, 6)
unsigned int idx_x = i_x * 2;
unsigned int idx_y = i_y * 2;

// Вычисляем интерполированное макро-среднее по вашей формуле
float macro_mid = ((k_S * (9 + idx_x) + k_LR * (7 - idx_x)) * (9 + idx_y) +
    (k_TB * (9 + idx_x) + k_LRTB * (7 - idx_x)) * (7 - idx_y)) / 256.0f;

unsigned int x3 = x / 8;
unsigned int y3 = y / 8;
if (x3 >= w3) x3 = w3 - 1;
if (y3 >= h3) y3 = h3 - 1;
float global_mid = p3[y3 * w3 + x3]; // Глобальный фон макроблока 8x8

// 2. ВЫЧИСЛЕНИЕ ДИСПЕРСИИ И ЛОКАЛЬНОГО КОНТРАСТА
float delta = src_gray - macro_mid;

```

```

// Внутренний сдвиг локальной разницы (аналог вашего макробарьера p4)
float local_dispersion = abs(macro_mid - global_mid);

// Защитный знаменатель против клякс и деления на ноль из 2011 года
float denom = local_dispersion + (12.75f / 255.0f); // Перевели 12.75 в диапазон [0.0 - 1.0]
DirectX
float local_contrast = delta / denom;

// Жесткое ограничение диапазона среза, как в HLSL шейдере
if (local_contrast < -1.5f) local_contrast = -1.5f;
if (local_contrast > 1.5f) local_contrast = 1.5f;

// 3. ФИНАЛЬНЫЙ СИНТЕЗ И КОРРЕКЦИЯ ДЕФЕКТОСКОПА
float out_gray = macro_mid + local_contrast * (local_dispersion * 1.8f);

// Нормализация яркости в стандартный диапазон [0.0, 255.0]
if (out_gray < 0.0f) out_gray = 0.0f;
if (out_gray > 255.0f) out_gray = 255.0f;

// Честная гамма-коррекция 2.2 силами центрального процессора
float norm_gray = out_gray / 255.0f;
float corrected_gray = pow(norm_gray, 1.0f / 2.2f);

// Проявление скрытых микроконтуров
corrected_gray = corrected_gray * 1.05f - 0.02f;
if (corrected_gray < 0.0f) corrected_gray = 0.0f;
if (corrected_gray > 1.0f) corrected_gray = 1.0f;

// Собираем готовый пиксель обратно в формат BGRA для вывода шторки
BYTE final_byte = (BYTE)(corrected_gray * 255.0f);
pOutPixels[idx] = (0xFF000000) | (final_byte << 16) | (final_byte << 8) | final_byte;
}
}

```

```

// Чистое освобождение динамической памяти из ОЗУ компьютера
delete[] p1;
delete[] p2;
delete[] p3;

return S_OK;
}

// Главная точка входа Windows-приложения
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam)
{
    switch (uMsg)
    {
        case WM_HSCROLL:
        {
            SCROLLINFO si = {};
            si.cbSize = sizeof(SCROLLINFO);
            si.fMask = SIF_ALL;
            GetScrollInfo(hwnd, SB_HORZ, &si);

            int oldPos = si.nPos;
            switch (LOWORD(wParam))
            {
                case SB_LINELEFT: si.nPos -= 20; break; // Кликнули на левую стрелочку
                case SB_LINERIGHT: si.nPos += 20; break; // Кликнули на правую стрелочку
                case SB_PAGELEFT: si.nPos -= si.nPage; break;
                case SB_PAGERIGHT: si.nPos += si.nPage; break;
                case SB_THUMBPOSITION: si.nPos = si.nTrackPos; break; // Реагируем только на отпускание
                // МЫШИ
            }
        }
    }
}

```

```

// Зажимаем позицию в физические рамки картинки
if (si.nPos < 0) si.nPos = 0;
if (si.nPos > (int)(g_currentImgWidth - si.nPage)) si.nPos = (int)(g_currentImgWidth - si.nPage);

if (si.nPos != oldPos)
{
    g_scrollX = si.nPos;

    // ВАША ФУНДАМЕНТАЛЬНАЯ ФОРМУЛА ДОЛИ:
    float denominatorX = (float)(si.nMax - si.nMin);
    //g_scrollRatioX = (denominatorX > 0.0f) ? (float)(si.nPos - si.nMin) / denominatorX : 0.0f;
    g_scrollRatioX = (float)(si.nPos - si.nMin) / (float)(si.nMax - si.nMin - si.nPage);

    SetScrollInfo(hwnd, SB_HORZ, &si, TRUE);

    Render();
}
}

g_bNeedsUpdate = true;
return 0;

case WM_VSCROLL:
{
    SCROLLINFO si = {};
    si.cbSize = sizeof(SCROLLINFO);
    si.fMask = SIF_ALL;
    GetScrollInfo(hwnd, SB_VERT, &si);

    int oldPos = si.nPos;
    switch (LOWORD(wParam))
    {
        case SB_LINEUP: si.nPos -= 20; break; // Кликнули на верхнюю стрелочку
        case SB_LINEDOWN: si.nPos += 20; break; // Кликнули на нижнюю стрелочку
    }
}

```

```

        case SB_PAGEUP: si.nPos -= si.nPage; break;

        case SB_PAGEDOWN: si.nPos += si.nPage; break;

        case SB_THUMBPOSITION: si.nPos = si.nTrackPos; break; // Реагируем только на отпускание
        мыши
    }

    if (si.nPos < 0) si.nPos = 0;

    if (si.nPos > (int)(g_currentImgHeight - si.nPage)) si.nPos = (int)(g_currentImgHeight - si.nPage);

    if (si.nPos != oldPos)
    {
        g_scrollY = si.nPos;

        // ВАША ФУНДАМЕНТАЛЬНАЯ ФОРМУЛА ДОЛИ:
        float denominatorY = (float)(si.nMax - si.nMin);
        //g_scrollRatioY = (denominatorY > 0.0f) ? (float)(si.nPos - si.nMin) / denominatorY : 0.0f;
        g_scrollRatioY = (float)(si.nPos - si.nMin) / (float)(si.nMax - si.nMin - si.nPage);

        SetScrollInfo(hwnd, SB_VERT, &si, TRUE);

        Render();
    }

}

g_bNeedsUpdate = true;

return 0;

case WM_SIZE:
{
    // 1. Измеряем реальные физические размеры окна
    g_wndW = (float)LOWORD(lParam);
    g_wndH = (float)HIWORD(lParam);

    // Обновляем положение строки статуса, передавая живые параметры

```

```

if (g_hStatusWnd != NULL)
{
    SendMessage(g_hStatusWnd, WM_SIZE, wParam, lParam);

    // Получаем высоту статус-бара, чтобы ползунки сели строго НАД ним
    RECT rS = {};
    GetWindowRect(g_hStatusWnd, &rS);
    g_wndH -= (float)(rS.bottom - rS.top);
}

// === НАЧАЛО ОДНОГО КОРОТКОГО ШАГА: ЛОГИКА МАСШТАБИРОВАНИЯ ИЗ КОММИТА ===
// Если выбран режим "Вписать в окно" (по умолчанию)
if (g_bFitToWindow)
{
    // В режиме "Вписать в окно" настраиваем выюпорт vpWin под текущие размеры рамы
    экрана
    vpWin.Width = g_wndW;
    vpWin.Height = g_wndH;
    vpWin.TopLeftX = 0.0f;
    vpWin.TopLeftY = 0.0f;
}
else
{
    // В режиме 1:1 фиксируем выюпорт vpWin под реальные 12 мегапикселей кадра
    vpWin.Width = (float)g_currentImgWidth;
    vpWin.Height = (float)g_currentImgHeight;
}

if (g_pSwapChain)
{
    g_pSwapChain->ResizeBuffers(0, (UINT)LOWORD(lParam), (UINT)HIWORD(lParam),
    DXGI_FORMAT_UNKNOWN, 0);
}

```

```

ID3D11Texture2D* pBackBuffer = NULL;

g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);

if (pBackBuffer) {
    g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
    pBackBuffer->Release();
}
}

g_bNeedsUpdate = true;
}

break;

```

```

// === НОВЫЙ КОД: ОБРАБОТКА НАЖАТИЙ НА ПУНКТЫ МЕНЮ ===

case WM_COMMAND:

    // В wParam система Windows передает ID нажатого пункта меню

    switch (LOWORD(wParam))
    {

    case IDM_VIEW_CPU:

        // 1. Инвертируем глобальный флаг

        g_bUseCPUProcessing = !g_bUseCPUProcessing;

        // 2. Ставим галочку в меню, используя точный идентификатор кнопки

        CheckMenuItem(GetMenu(hwnd), IDM_VIEW_CPU, g_bUseCPUProcessing ? MF_CHECKED :
MF_UNCHECKED);

        // 3. Командуем окну немедленно перерисовать кадр

        InvalidateRect(hwnd, NULL, FALSE);

        g_bNeedsUpdate = true;

        return 0;

```

```

// === НАСТОЯЩЕЕ ВНЕДРЕНИЕ: ОБРАБОТКА НАЖАТИЙ НА МАСШТАБЫ ДЕФЕКТΟΣКОПА ===
case IDM_ZOOM_FIT: g_bFitToWindow = true; g_zoomDivider = 1.0f; goto my_reset_scroll;
case IDM_ZOOM_1: g_bFitToWindow = false; g_zoomDivider = 1.0f; goto my_reset_scroll;
case IDM_ZOOM_2: g_bFitToWindow = false; g_zoomDivider = 2.0f; goto my_reset_scroll;
case IDM_ZOOM_4: g_bFitToWindow = false; g_zoomDivider = 4.0f; goto my_reset_scroll;
case IDM_ZOOM_8: g_bFitToWindow = false; g_zoomDivider = 8.0f; goto my_reset_scroll;
case IDM_ZOOM_16: g_bFitToWindow = false; g_zoomDivider = 16.0f; goto my_reset_scroll;

my_reset_scroll:
{
    // 1. Возвращаем скроллинг в исходный левый верхний угол кадра
    g_scrollX = 0; g_scrollY = 0;
    g_scrollRatioX = 0.0f; g_scrollRatioY = 0.0f;

    // 2. АВТОМАТИЧЕСКАЯ РАССТАНОВКА ГАЛОЧЕК (V) В МЕНЮ ОКНА
    HMENU hViewMenuRef = GetSubMenu(GetMenu(hwnd), 1); // Находим наше меню "Вид"
    (индекс 1)
    if (hViewMenuRef)
    {
        HMENU hZoomMenuRef = GetSubMenu(hViewMenuRef, 1); // Находим подменю "Масштаб"
        внутри "Вида"
        if (hZoomMenuRef)
        {
            // Сбрасываем старые птички со всех 6 пунктов
            CheckMenuItem(hZoomMenuRef, IDM_ZOOM_FIT, MF_UNCHECKED);
            CheckMenuItem(hZoomMenuRef, IDM_ZOOM_1, MF_UNCHECKED);
            CheckMenuItem(hZoomMenuRef, IDM_ZOOM_2, MF_UNCHECKED);
            CheckMenuItem(hZoomMenuRef, IDM_ZOOM_4, MF_UNCHECKED);
            CheckMenuItem(hZoomMenuRef, IDM_ZOOM_8, MF_UNCHECKED);
            CheckMenuItem(hZoomMenuRef, IDM_ZOOM_16, MF_UNCHECKED);

            // Включаем галочку строго на выбранном оператором режиме!
            UINT activeID = IDM_ZOOM_FIT;

```

```

        if (!g_bFitToWindow)
        {
            if (g_zoomDivider == 1.0f) activeID = IDM_ZOOM_1;
            if (g_zoomDivider == 2.0f) activeID = IDM_ZOOM_2;
            if (g_zoomDivider == 4.0f) activeID = IDM_ZOOM_4;
            if (g_zoomDivider == 8.0f) activeID = IDM_ZOOM_8;
            if (g_zoomDivider == 16.0f) activeID = IDM_ZOOM_16;
        }

        CheckMenuItem(hZoomMenuRef, activeID, MF_CHECKED);
    }
}

```

// 3. СИНХРОНИЗАЦИЯ ПОЛЗУНКОВ ПОД ВЫБРАННЫЙ МАСШТАБ

```

UINT currentMaxX = g_bFitToWindow ? (UINT)g_wndW : (UINT)((float)g_currentImgWidth /
g_zoomDivider);

```

```

UINT currentMaxY = g_bFitToWindow ? (UINT)g_wndH : (UINT)((float)g_currentImgHeight /
g_zoomDivider);

```

```

SCROLLINFO zoomSiX = {};

zoomSiX.cbSize = sizeof(SCROLLINFO);

zoomSiX.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;

zoomSiX.nMin = 0;

zoomSiX.nMax = (currentMaxX > 0) ? currentMaxX : 1;

zoomSiX.nPage = (UINT)g_wndW;

zoomSiX.nPos = 0;

SetScrollInfo(hwnd, SB_HORZ, &zoomSiX, TRUE);

```

```

SCROLLINFO zoomSiY = {};

zoomSiY.cbSize = sizeof(SCROLLINFO);

zoomSiY.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;

zoomSiY.nMin = 0;

zoomSiY.nMax = (currentMaxY > 0) ? currentMaxY : 1;

zoomSiY.nPage = (UINT)g_wndH;

```

```

zoomSiY.nPos = 0;

SetScrollInfo(hwnd, SB_VERT, &zoomSiY, TRUE);

// 4. Перерисовываем экран
InvalidateRect(hwnd, NULL, FALSE);

g_bNeedsUpdate = true;

return 0;
}

case IDM_FILE_EXIT:

    // Если нажали "Выход" — закрываем окно программы
    DestroyWindow(hwnd);

    return 0;

case IDM_HELP_ABOUT:

    // Если нажали "О программе" — показываем красивое информационное окно
    MessageBoxW(hwnd,

        L"Шейдерный Видеоплеер-Дефектоскоп v2026\n\n"

        L"Реализован пятипроходный вычислительный конвейер.\n"

        L"Алгоритмы фильтрации локального контраста и дисперсии макроблоков 4x4 и 8x8.\n\n"

        L"Разработчик математического ядра: Курчанов А. Ф.",

        L"О программе",

        MB_OK | MB_ICONINFORMATION);

    return 0;

case IDM_FILE_OPEN_VIDEO:

    // Вызываем Проводник в режиме фильтрации видео (передаем true)
    if (OpenFileDialog(hwnd, true))
    {
        // Если файл выбран успешно — выведем его путь на экран для проверки

        //MessageBoxW(hwnd, g_szSelectedFilePath, L"Видео поток успешно подключен", MB_OK |
        MB_ICONINFORMATION);

```

```

        // TODO: Здесь на следующем шаге мы запустим инициализацию Media Foundation
    }

    return 0;

case IDM_FILE_OPEN_IMAGE:

    // 1. Распакиваем Проводник для выбора картинки
    if (OpenFileDialog(hwnd, false))
    {
        // 2. Если файл выбран — загружаем его пиксели в Point-текстуру через WIC
        if (SUCCEEDED(LoadTextureFromFile(g_szSelectedFilePath)))
        {
            // 3.

            // Принудительно вызываем Render прямо сейчас,
            // чтобы протолкнуть новые SRV-интерфейсы в GPU!

            Render();

            wchar_t statusBuf[256];

            // Подставляем ваши глобальные переменные ширины и высоты кадра плеера
            wprintf(statusBuf, L"Файл успешно загружен. Габариты кадра: %d x %d пикселей.",
g_currentImgWidth, g_currentImgHeight);

            // Отправляем этот текст в нижний статус-бар!
            SendMessage(g_hStatusWnd, SB_SETTEXT, 0, (LPARAM)statusBuf);

            //RenderSplit(); // Переключили плеер в режим сравнения "До / После"

            //MessageBoxW(hwnd, L"Удалось декодировать файл изображения через WIC.", L"НЕТ
ОШИБОК", MB_OK | MB_ICONERROR);
        }
        else
        {
            MessageBoxW(hwnd, L"Не удалось декодировать файл изображения через WIC.",
L"Ошибка дефектоскопа", MB_OK | MB_ICONERROR);
        }
    }
}

```

```
    return 0;

case IDM_FILE_SAVE_RESULT:

case IDM_VIEW_SPLIT:

case IDM_VIEW_RESULT_ONLY:

    // Для остальных пунктов пока выведем временную заглушку, чтобы видеть отклик
    MessageBoxW(hwnd, L"Этот блок конвейера находится в режиме подключения данных.",
L"Информация", MB_OK | MB_ICONINFORMATION);

    return 0;

}

break;

// === КОНЕЦ НОВОГО КОДА ===

case WM_DESTROY:

    PostQuitMessage(0);

    return 0;

}

return DefWindowProc(hwnd, uMsg, wParam, lParam);

}
```